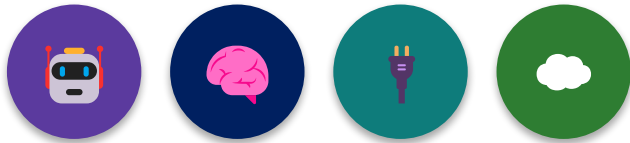


SUPPLEMENTARY · MODULE 12

Copilot, OpenShift AI, and MCP

Use Copilot, OpenShift AI, and MCP as force multipliers on the same architecture: generate, ground, and govern so banking code stays reviewable.



Day 5 Objectives and Capstone Finish Line



DAY 5 OBJECTIVES



1. Validate the Deployed Platform

Perform end-to-end validation of all capstone services in OpenShift.



2. Observe and Optimize

Analyze metrics, logs, and traces. Tune alerts and dashboards.



3. Ensure Security and Compliance

Review security configurations, secrets, and access controls.



4. Validate Resilience

Test failure scenarios and verify self-healing and recovery.



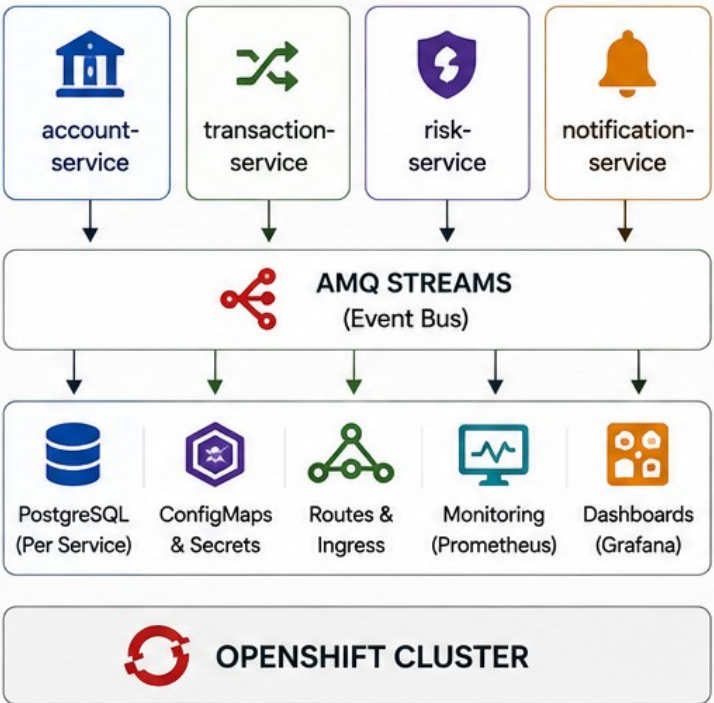
5. Finalize and Document

Capture results, create final report, and document key learnings.



CAPSTONE FINISH LINE

DEPLOYED CAPSTONE PLATFORM



FINISH LINE CRITERIA

- ✓ All services deployed and running
- ✓ Routes accessible and functional
- ✓ Health checks passing
- ✓ Events flowing through AMQ Streams
- ✓ Metrics, logs, and dashboards active
- ✓ Alerts configured and verified
- ✓ Security, secrets, and access controls validated
- ✓ Resilience tests completed successfully
- ✓ Documentation and final report completed

CAPSTONE OUTCOMES



Production-Ready Microservices Platform

Deployed and validated on OpenShift



Observable and Monitored

Metrics, logs, and alerts fully operational



Secure and Compliant

Security best practices implemented



Resilient and Reliable

Failure scenarios tested and recovered



Well Documented and Reported

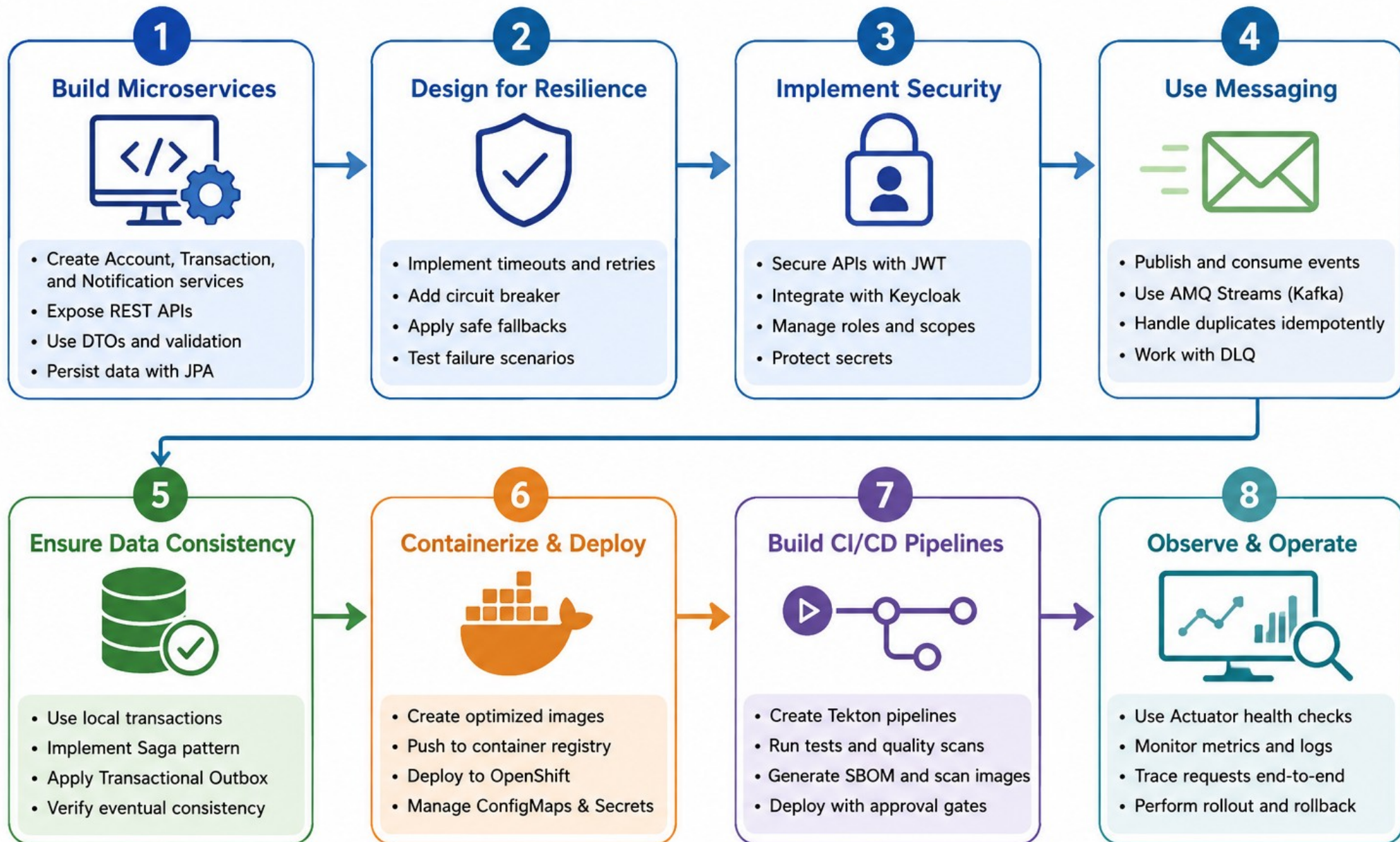
Findings, runbooks, and lessons learned captured



Capstone Complete

You've built, deployed, and validated a cloud-native microservices platform!

What Participants Will Do Hands-On



Responsible AI in Banking Workflows



INTRODUCTION TO GITHUB COPILOT · GITHUB COPILOT PRODUCT CAPABILITIES



WHAT COPILOT HELPS BUILD

- REST controllers, service layer methods, repository interfaces, DTO classes
- Validation rules, unit tests, refactoring suggestions, API documentation



THE KEY MESSAGE

- Generated code must still meet enterprise standards for security, correctness, and compliance
- Copilot is a development assistant, not a replacement for software design



CORE CAPABILITIES

- Complete code, generate methods, create tests
- Explain unfamiliar code, suggest refactoring, generate documentation, draft configuration files

KEY TAKEAWAY Copilot generates boilerplate, tests, and documentation, but engineering judgment, review, and testing still belong to the developer. • Understanding the full capability surface helps a team decide which Copilot mode fits which part of the Spring Boot workflow.

HOW AI CODING ASSISTANTS GENERATE SUGGESTIONS · GITHUB COPILOT IN THE IDE

· COPILOT CHAT AND INLINE ASSISTANCE



WHAT SHAPES A SUGGESTION

- Existing project files and open editor tabs
- Comments, method names, class names, chat prompts, and selected code
- The quality of suggestions depends heavily on context and prompt clarity



IDE-INTEGRATED WORKFLOW

- Suggestions appear inline as ghost text while coding
- Accept, reject, or cycle through alternative suggestions without leaving the editor



TWO INTERACTION MODES

- **Copilot Chat:** conversational, multi-turn requests for larger code units or explanations
- **Inline assistance:** fast, in-editor suggestions as you type, accepted with a keystroke

KEY TAKEAWAY Existing project files, open editor tabs, comments, method and class names, and chat prompts all shape what Copilot suggests next. • Copilot proposes code directly at the cursor as you type, accepted or rejected line by line or block by block. • Chat suits larger, conversational requests with back-and-forth refinement; inline completion suits fast, in-flow micro-suggestions.

UNDERSTANDING REPOSITORY CONTEXT · PROMPT ENGINEERING FOR SOFTWARE DEVELOPMENT



WHY REPOSITORY CONTEXT MATTERS

- Copilot infers project conventions from surrounding code, not just the current file
- Keeping related files open improves the relevance of generated suggestions



WHAT GOOD PROMPTS DESCRIBE

- The framework, the class or method needed, existing project patterns
- Required inputs and outputs, validation rules, error handling and testing expectations

KEY TAKEAWAY A repository with consistent naming, layering, and DTO conventions gives Copilot the strongest signal to follow those same conventions. • Good coding prompts describe the framework, the class or method needed, existing patterns, inputs/outputs, validation, and error handling.

WRITING CLEAR CODING PROMPTS · PROVIDING CONSTRAINTS AND ACCEPTANCE CRITERIA

VAGUE PROMPT

- "Create customer API." -- no endpoints, DTOs, validation, or status codes specified

BETTER PROMPT

- Name the controller, base path, methods, DTOs, @Valid usage, and expected status codes
- **State explicitly:** "Do not put business logic in the controller. Follow constructor injection."

PROMPT COMPONENTS

- **Role:** "Act as a Spring Boot developer"
Task: "Create a controller"
- **Context:** existing DTOs and exception handling
Constraints: "Use constructor injection"
- **Output:** "Return only the Java class"
Quality: "Include validation and HTTP status codes"

KEY TAKEAWAY A vague prompt leaves endpoints, validation, and status codes to chance; a specific prompt gives Copilot enough structure to draft correctly. • Role, task, context, constraints, output format, and quality expectations are the six ingredients of a well-formed coding prompt.

USING EXAMPLES AND EXISTING CODE AS CONTEXT · ITERATIVE PROMPTING AND REFINEMENT



WHY EXAMPLES WORK

- A concrete example transmits conventions that are hard to fully describe in words
- Pointing Copilot at an existing, well-reviewed class as a pattern improves consistency



THE ITERATIVE LOOP

1. Generate initial code
2. Review gaps
3. Ask for improvements
4. Add missing constraints
5. Refine to match project standards

KEY TAKEAWAY An example of an existing controller or service teaches Copilot your naming, layering, and error-handling conventions by demonstration.

- Generate initial code, review gaps, ask for improvements, add missing constraints, and refine to match project standards -- five iterative steps.

GENERATING A SPRING BOOT PROJECT STRUCTURE · GENERATING REST CONTROLLERS · GENERATING SERVICE-LAYER LOGIC



SCAFFOLDING A NEW SERVICE

- **Prompt for the full package structure:** controller, service, repository, model, dto, exception
- Review that generated classes land in the correct package for their responsibility



WHAT TO SPECIFY IN THE PROMPT

- Collaborators (repository, event publisher), method signatures, business rules
- Exceptions to throw, @Transactional placement, event publication timing



EXPECTED CONTROLLER SHAPE

- @RestController, base @RequestMapping, method mappings, DTO request/response types
- @Valid validation trigger, @PathVariable/@RequestBody, appropriate ResponseEntity usage



COMMON GENERATED PROBLEMS

- Missing business rules, incorrect exception handling, events published before save succeeds
- Forgotten @Transactional, excessive duplicated mapping code



COMMON GENERATED PROBLEMS

- Business logic in the controller, entities returned directly, missing @Valid
- 200 OK returned for create instead of 201 Created; invented service methods

KEY TAKEAWAY Describing the target package layout up front keeps Copilot's generated classes landing in the right layer from the very first draft. • @RestController, DTOs, @Valid, and correct status codes define a good generated controller; business logic in the controller is a common defect. • Prompting for exceptions, transaction annotations, and event publication order explicitly prevents Copilot from guessing at business rules.

GENERATING REPOSITORY COMPONENTS · GENERATING DTOS AND VALIDATION RULES · GENERATING EXCEPTION HANDLING



EXPECTED REPOSITORY SHAPE

- extends JpaRepository<Entity, Long>, Optional for single-record lookups
- findBy... derived methods and existsBy... checks matching real entity fields



PROMPTING FOR DTOS

- Name the request and response DTO fields explicitly
- **State validation rules:** required fields, format constraints, ranges, date rules



COMMON GENERATED PROBLEMS

- Field names that don't exist, overly broad methods, missing pagination, missing Optional



GENERATING EXCEPTION HANDLING

- Custom exception classes (e.g., CustomerNotFoundException) extending RuntimeException
- @RestControllerAdvice with @ExceptionHandler methods mapping to correct HTTP status codes

KEY TAKEAWAY Providing entity field names in the prompt is what keeps Copilot's derived query methods from referencing properties that don't exist. • @NotBlank, @Email, @Size, and @Past map cleanly onto request DTO fields once the business rules are stated in the prompt. • Custom exceptions plus one global exception handler give every generated controller the same consistent, reviewable error contract.

GENERATING CONFIGURATION COMPONENTS · GENERATING DATABASE QUERIES · GENERATING MESSAGING PRODUCERS AND CONSUMERS



GENERATING CONFIGURATION

- @ConfigurationProperties classes, application.yml sections, profile-specific overrides
- Never include real secrets, passwords, or environment-specific URLs in the prompt itself



PROMPTING FOR CUSTOM QUERIES

- "Create a @Query method to find recent transactions by accountId and status, ordered by date descending, returning Page<T>"
- Review generated SQL or JPQL behavior and confirm indexes exist for the queried fields



GENERATING MESSAGING CODE

- Publisher components using RabbitTemplate or KafkaTemplate with a defined event DTO
- @RabbitListener or @KafkaListener consumer methods with duplicate-event handling reviewed manually

KEY TAKEAWAY Asking for a configuration class with clear property names is safe; asking for real credentials or environment values in the prompt is not.

- Requesting a paginated, ordered @Query method explicitly avoids Copilot generating an inefficient or overly long derived query name.
- Copilot can draft a RabbitTemplate publisher or @KafkaListener consumer, but duplicate-handling logic must be reviewed, not assumed.

GENERATING UNIT TESTS · GENERATING INTEGRATION TESTS · GENERATING TEST DATA AND EDGE CASES



PROMPTING FOR UNIT TESTS

- "Generate JUnit 5 and Mockito tests for CustomerService.createCustomer: success, duplicate email, repository save failure"
- Verify mock interactions (repository, event publisher) with clear test method names



GENERATING INTEGRATION TESTS

- **Controller tests validating status codes:** 201 Created, 400 Bad Request, 404 Not Found
- Repository tests validating persistence behavior against a real or embedded database



PROMPTING FOR EDGE CASES

- Ask explicitly for negative opening balance, duplicate account number, missing customer ID
- Add missing edge cases manually when the generated set only covers the happy path

KEY TAKEAWAY Prompting for specific scenarios (success, duplicate, not-found) produces far stronger tests than a generic "write tests for this class" request. • A generated integration test that mocks the very layer it should be testing gives a false sense of coverage. • Negative values, boundary conditions, duplicate keys, and missing required fields rarely appear unless the prompt asks for them by name.

USING COPILOT FOR DEBUGGING · USING COPILOT FOR CODE EXPLANATION



USING COPILOT WHEN DEBUGGING

- Paste the stack trace or failing test output and ask for likely causes
- Ask Copilot to explain unfamiliar framework behavior surfacing in the error



USING COPILOT FOR EXPLANATION

- Ask what a Spring annotation, design pattern, or unfamiliar method actually does
- Cross-check the explanation against official documentation for anything safety-critical

KEY TAKEAWAY Pasting an exception and asking "why might this happen" is often faster than searching, but the suggested fix still needs validation against real behavior. • A clear explanation of an annotation or pattern is a starting point for understanding, not a substitute for verifying it against the real codebase.

USING COPILOT FOR REFACTORING · IMPROVING PERFORMANCE WITH COPILOT · GENERATING API DOCUMENTATION



SAFE REFACTORING WORKFLOW

1. Ensure tests exist
2. Select a focused code section
3. Ask for a refactoring plan or draft
4. Apply small changes
5. Run tests
6. Review behavior and API compatibility



PERFORMANCE-FOCUSED PROMPTS

- Ask Copilot to identify N+1 query risks, missing indexes, or unnecessary eager loading
- Measure query count and latency before and after applying any suggested change



PROMPTING FOR DOCUMENTATION

- "Generate README documentation including overview, endpoints, examples, errors, environment variables, run instructions"
- **Explicitly instruct:** "Do not include secrets" and verify every endpoint against real code

KEY TAKEAWAY Ensure tests exist, refactor a focused section, apply small changes, and run tests -- the same discipline whether a human or Copilot writes the diff. • A performance suggestion is a hypothesis until profiling data confirms it actually improved latency or query count. • A documentation draft can include fake endpoints or stale examples unless someone confirms it against the actual running service.

GENERATING CONTAINER AND PIPELINE FILES · REVIEWING AI-GENERATED CODE · DETECTING INCORRECT OR INVENTED APIS



REVIEWING GENERATED INFRASTRUCTURE FILES

- Confirm the base image is enterprise-approved and pinned to a specific version
- Confirm no secrets are embedded and the container runs as a non-root user



REVIEW CHECKLIST

- Correct package/class names, constructor injection, no business logic in controllers
- DTOs instead of entities, validation annotations, correct HTTP status codes
- No hardcoded secrets, no unsafe logging, tests for success and failure cases

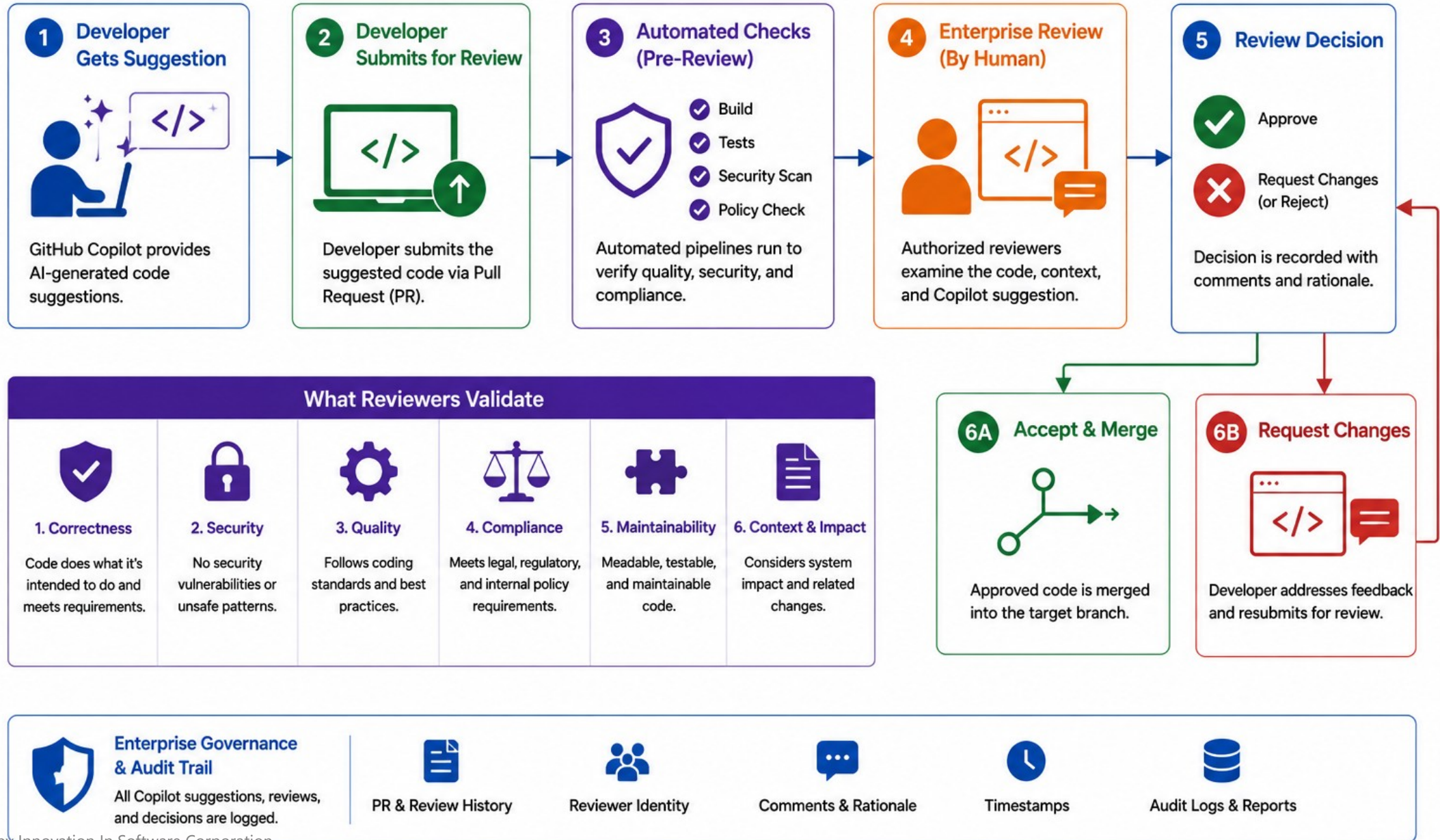


COMMON HALLUCINATION PATTERNS

- Calling a method that does not exist on the actual class
- Referencing a library not present in the project's dependencies
- Describing a framework feature incorrectly, or generating config for the wrong version

KEY TAKEAWAY Approved base images, non-root users, and no embedded secrets are non-negotiable checks on any Copilot-drafted Dockerfile or pipeline stage. • Package names, constructor injection, DTO usage, validation, status codes, secrets, and tests -- a nine-point checklist for every generated class. • Calling a nonexistent method, referencing an unapproved library, or misdescribing a framework feature are all classic hallucination patterns.

GitHub Copilot - Enterprise Review-Before-Accept



SECURITY RISKS IN AI-GENERATED CODE · INTELLECTUAL PROPERTY AND LICENSING CONSIDERATIONS



COMMON SECURITY DEFECTS

- Hardcoded secrets, weak input validation, unsafe SQL construction
- Excessive or sensitive logging, missing authorization checks



WHAT TO VERIFY

- Is the suggested library on the enterprise's approved dependency list?
- Is its license compatible with how the software will be distributed or sold?

KEY TAKEAWAY Hardcoded secrets, weak validation, unsafe SQL construction, and excessive logging are the security defects most likely to slip through in generated code. • Verifying license compatibility and approval status before accepting a generated dependency protects the enterprise from downstream legal risk.

PROTECTING SENSITIVE DATA IN PROMPTS · ENTERPRISE GOVERNANCE FOR GITHUB COPILOT

· MEASURING COPILOT PRODUCTIVITY



NEVER INCLUDE IN PROMPTS

- Real customer data, account numbers, passwords, tokens, API keys
- Internal security details or confidential business logic where policy restricts sharing



RESPONSIBLE USE PRINCIPLES

- Human review, security awareness, testing discipline, policy compliance
- Data protection, traceability, architecture alignment



BETTER PRODUCTIVITY SIGNALS

- Time from task start to merged, reviewed pull request
- Defect rate in Copilot-assisted code versus baseline, review time per change

KEY TAKEAWAY Anything typed into a prompt can be logged or retained -- treat prompt content with the same care as a support ticket or a commit message.

- Human review, security scanning, pull request review, and CI quality gates apply to generated code exactly as they do to any other change.
- Cycle time, defect rate, and review time tell a more honest productivity story than raw suggestion-acceptance counts ever could.

COPILOT BEST PRACTICES FOR ENTERPRISE TEAMS · HUMAN REVIEW BEFORE ACCEPTING SUGGESTIONS



TEAM BEST PRACTICES

- Use AI for focused tasks; review generated code thoroughly; run tests and scans
- Avoid sensitive prompt content; follow enterprise dependency policy; keep PR review requirements



WHY HUMAN REVIEW IS NON-NEGOTIABLE

- Copilot is an assistant, not an accountable engineer
- The developer and engineering team remain responsible for final code quality

KEY TAKEAWAY A short list of team-wide habits keeps Copilot's speed advantage from ever coming at the cost of enterprise engineering standards. • Accepting a suggestion is not the same as approving it -- that decision belongs to the developer who will be accountable for it in production.

Generative AI vs. Agentic AI

From content generation to goal-driven action.

Both are powerful. They solve different problems.



Generative AI

Creates content when prompted.



Agentic AI

Acts toward goals autonomously.

ASPECT	GENERATIVE AI	AGENTIC AI	WHEN TO USE WHICH
Purpose	Generate content, ideas, or predictions based on input. <i>Answers questions. Creates outputs.</i>	Achieve goals by taking actions, using tools, and adapting to feedback. <i>Gets things done. Delivers outcomes.</i>	Use Generative AI when you need to: ✓ Create or summarize content ✓ Write code snippets or documentation ✓ Translate or explain complex topics ✓ Brainstorm ideas and alternatives ✓ Rapidly prototype text or images
How it works	Responds to prompts with learned patterns from training data. <i>Stateless between interactions.</i>	Plans, reasons, takes actions, observes results, and adapts. <i>Stateful, iterative, and goal-directed.</i>	
Autonomy	Human-in-the-loop for every request. No persistent autonomy. <i>You ask, it responds.</i>	Can operate with a degree of autonomy within defined boundaries. <i>You set the goal, it plans and acts.</i>	
Capabilities	Text, code, images, summaries, translation, brainstorming, Q&A, and more. <i>Great at creation.</i>	Use tools, call APIs, access data, take actions, coordinate workflows, learn from feedback. <i>Great at execution.</i>	Use Agentic AI when you need to: ✓ Achieve multi-step goals ✓ Orchestrate systems and APIs ✓ Automate business processes ✓ Monitor, adapt, and optimize ✓ Handle dynamic, real-world tasks
Examples	ChatGPT, GitHub Copilot (inline suggestions), DALL·E, Claude, Gemini. <i>Content generation tools.</i>	AutoGPT, Microsoft Copilot (agent mode), OpenAI Assistants API, LangChain agents. <i>Autonomous agents and copilots.</i>	
Strengths	• Fast, creative, and scalable content generation • Excellent for ideation and augmentation • Easy to use and prompt-driven	• Goal-oriented and outcome-focused • Can use tools and integrate with systems • Adapts to changing context and feedback	
Limitations	• No real-world actions or tool use • Can hallucinate or be factually incorrect • Requires human direction for every step	• More complex to design and govern • Higher risk if not properly bounded • Requires strong guardrails and observability	

THE CONTINUUM



THE BIG PICTURE

Generative AI helps you **create**.
Agentic AI helps you **accomplish**.
Together, they amplify human impact.



The future isn't one or the other—it's human + AI working together with the right tool for the right job.



Human Intent
Sets goals and defines success



AI Capability
Provides intelligence and automation

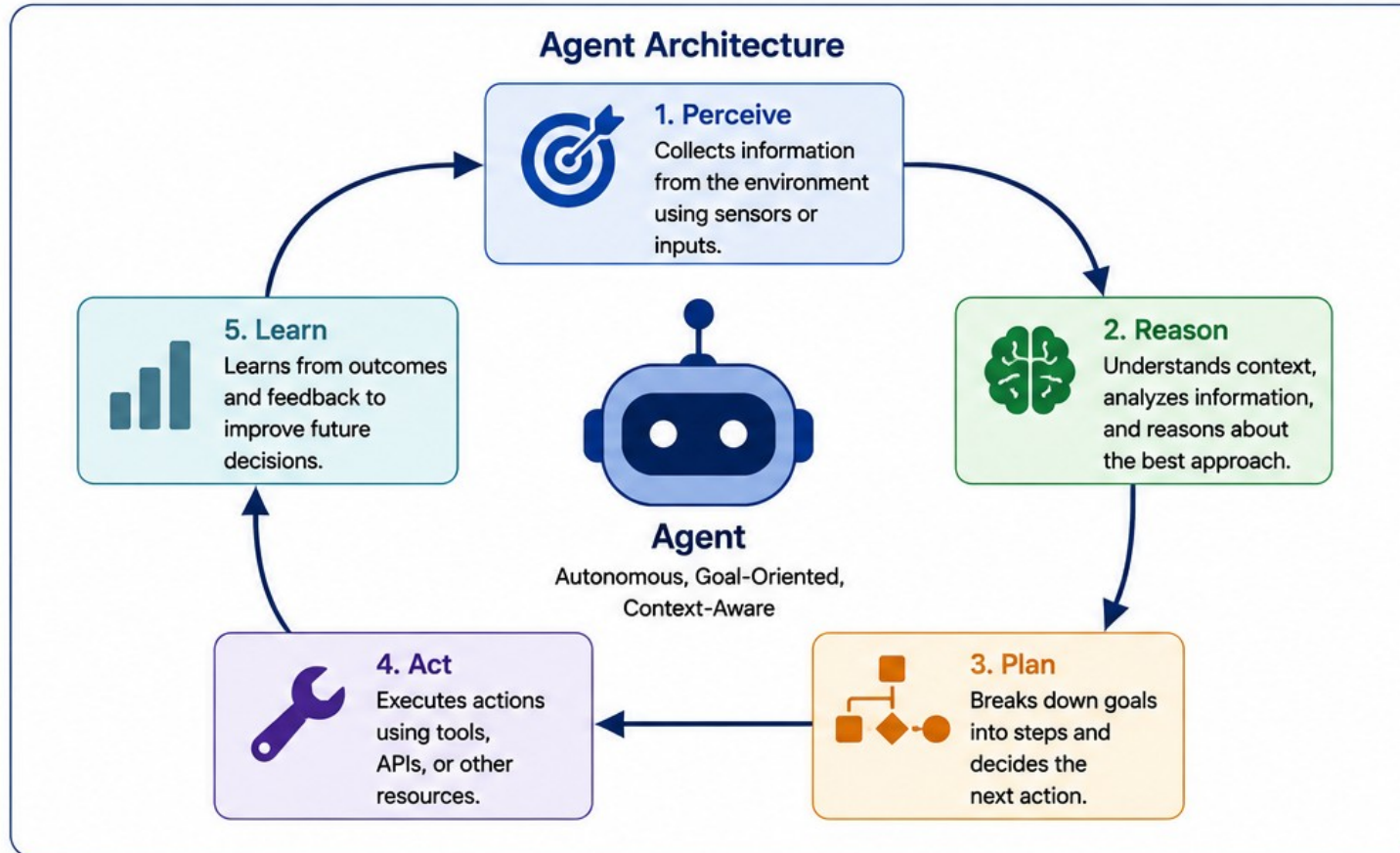


Governance
Ensures safety, ethics, and trust



Outcome
Delivers measurable business value

Agentic AI Concepts



Key Characteristics



Autonomy

Operates independently to achieve goals.



Goal-Oriented

Works towards defined objectives.



Perception

Observes and understands the environment.



Reasoning

Thinks, infers, and makes decisions.



Actionable

Executes actions using tools and interfaces.



Adaptive

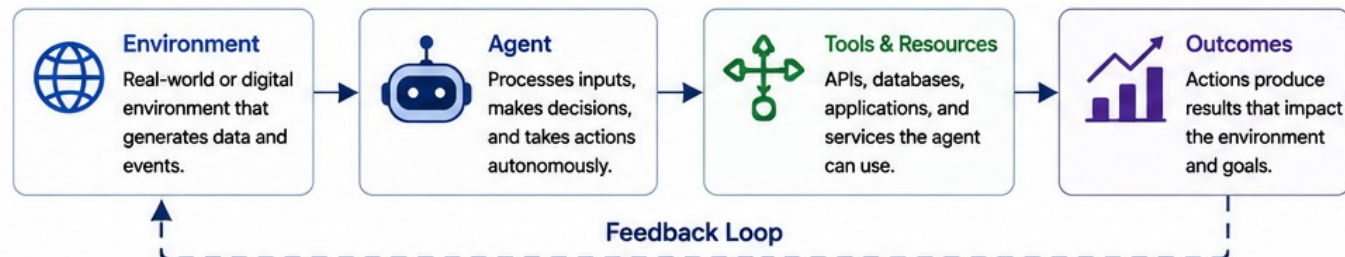
Learns and adapts from experience and feedback.



Robust & Resilient

Handles errors and changes in the environment.

How Agentic AI Works



Common Use Cases



Customer Support



Data Analysis & Insights



Software Development



E-Commerce Automation



Operations Optimization

WHAT MAKES AN AI SYSTEM AN AGENT · MODELS, AGENTS, TOOLS, AND ENVIRONMENTS · AGENT GOALS AND TASK DECOMPOSITION



AGENTIC WORKFLOW ELEMENTS

- Goal, context, plan, tools, memory or state
- API access, observations, decision points, human approval, final output



THE FOUR BUILDING BLOCKS

- **Model:** the reasoning engine (LLM) that interprets goals and plans steps
- **Agent:** the orchestrating layer around the model that manages tools, memory, and policy
- **Tools:** callable capabilities (APIs, functions) the agent can invoke
- **Environment:** the system the agent operates within, including its constraints



GOAL CLARITY MATTERS

- **Good:** "Summarize failed payment reasons for support review."
- **Too broad:** "Handle customer banking issues."
- A clear goal decomposes into concrete, tool-callable steps

KEY TAKEAWAY Goal, context, plan, tools, memory, and observations together turn a language model into something that can act, not just respond. • Separating these four concepts clarifies exactly where a Spring Boot API fits: as a tool the agent can call, not as the agent itself. • "Summarize failed payment reasons for support review" decomposes cleanly; "handle customer banking issues" does not.

PLANNING AND REASONING LOOPS · OBSERVATION, ACTION, AND FEEDBACK



THE REASONING LOOP

1. Plan the next action based on the goal and current state
2. Act by calling a tool or producing output
3. Observe the result
4. Replan or conclude



THE OBSERVE-ACT-FEEDBACK CYCLE

- **Action:** the agent calls a tool or takes a step toward its goal
- **Observation:** the agent reads the actual result, success or failure
- **Feedback:** the observation informs the next planning step

KEY TAKEAWAY This plan-act-observe loop is what separates a multi-step agent from a single-shot prompt-and-response interaction. • If a tool call fails or returns unexpected data, the agent must observe that honestly rather than proceeding as if it succeeded.

TOOL CALLING AND FUNCTION CALLING · AGENT MEMORY AND CONTEXT

· SHORT-TERM VS. LONG-TERM MEMORY



HOW TOOL CALLING WORKS

- The model is given a list of available tools with names, descriptions, and parameter schemas
- The model responds with a structured request to call one, which the framework executes
- The tool's result is returned to the model to continue reasoning



MEMORY VS. CONTEXT

- **Context:** the information available to the model for the current reasoning step
- **Memory:** information retained and retrievable across steps or sessions



TWO MEMORY TYPES

- **Short-term memory:** the current task's working context, often the conversation window
- **Long-term memory:** persisted facts, preferences, or history retrievable across sessions

KEY TAKEAWAY The model doesn't execute the tool itself -- it requests a call, the agent framework executes it, and the result flows back into the conversation.

- Without memory, an agent re-asks questions it already had answers to; without bounded context, it drowns in irrelevant history.
- A support agent's short-term memory holds this conversation's transaction IDs; long-term memory might recall a customer's past support history.

RETRIEVAL-AUGMENTED GENERATION · DETERMINISTIC WORKFLOWS VS. AUTONOMOUS AGENTS



HOW RAG WORKS

- A retrieval step searches an indexed knowledge base for relevant content
- Retrieved content is added to the model's context before it generates a response
- This grounds answers in real, current, trusted source material



WHEN TO USE EACH

- **Deterministic workflow:** the sequence of steps is known and fixed in advance
- **Autonomous agent:** the model decides which tools to call and in what order, based on context
- Prefer deterministic workflows when the process is well-understood and repeatable

KEY TAKEAWAY An AI assistant retrieving relevant policy documents before answering a support question is a direct application of RAG in a banking context.

- Deterministic workflows are predictable and easy to test; autonomous agents are flexible but require stronger guardrails and observability.

SINGLE-AGENT VS. MULTI-AGENT SYSTEMS · AGENT ORCHESTRATION PATTERNS · HUMAN-IN-THE-LOOP WORKFLOWS



SINGLE-AGENT VS. MULTI-AGENT

- **Single-agent:** one model handles the entire task end to end -- simpler to reason about
- **Multi-agent:** specialized agents each handle part of the workflow and coordinate results



COMMON ORCHESTRATION PATTERNS

- **Centralized orchestrator:** one controller directs each agent's turn explicitly
- **Decentralized coordination:** agents react to shared events or state independently



WHY HUMAN-IN-THE-LOOP MATTERS

- Adds a checkpoint before sensitive or irreversible actions take effect
- Keeps a human accountable for the final decision on regulated or high-impact outcomes

KEY TAKEAWAY A multi-agent design might have one agent gather data, another summarize it, and a third draft the customer-facing response.

- The orchestration pattern chosen determines how visible, debuggable, and controllable a multi-agent workflow will be in production.
- The assistant drafts a customer message; a support representative reviews and sends it -- the agent never communicates with the customer directly.

AGENT PERMISSIONS AND BOUNDARIES · DESIGNING SAFE AUTONOMOUS WORKFLOWS · ERROR HANDLING AND RECOVERY FOR AGENTS



AGENT BOUNDARY QUESTIONS

- What can the agent read? What can the agent change? Which tools are available?
- Which actions need approval? What data is masked? What is logged?



WHAT TO DEFINE BEFORE BUILDING

- Trigger, goal, allowed tools, data access scope
- Decision rules, human approval points, error handling, stop conditions, audit logging



RECOVERY OPTIONS

- Return a draft instead of sending, escalate to human review, create a case
- Retry safe tool calls only, stop and report missing data clearly

KEY TAKEAWAY Defining these boundaries explicitly before deployment is what prevents an agent from accidentally exceeding its intended authority. • Trigger, goal, allowed tools, data access scope, decision rules, approval points, error handling, and stop conditions -- design all eight up front. • Returning a draft instead of sending, escalating to human review, or retrying only safe operations are all valid recovery paths -- silence is not.

PREVENTING INFINITE AGENT LOOPS · AGENT EVALUATION AND TESTING · AGENT OBSERVABILITY



LOOP-PREVENTION CONTROLS

- Set a maximum number of reasoning/tool-call steps per task
- Detect repeated identical failures and stop instead of retrying indefinitely
- Apply an overall timeout budget for the entire workflow



WHAT TO TEST

- Normal cases, missing data, conflicting data, tool failure
- Unauthorized access, sensitive data handling, human approval flow, output format validation



WHAT TO OBSERVE

- User request, model call, tool calls, API latency, failure points
- Generated output, approval decisions, cost and token usage where applicable

KEY TAKEAWAY Maximum step counts, timeout budgets, and repeated-failure detection are the guardrails that keep an autonomous loop from running away. • An agent that has never been tested against a failing tool or an unauthorized request is an agent whose failure behavior is unknown. • If an agent's behavior can't be reconstructed after the fact, it can't be trusted to run unattended in front of real customers.

TRUST, RISK, AND RESPONSIBLE AI · PROMPT INJECTION AND TOOL ABUSE · DATA LEAKAGE AND PRIVACY RISKS



RESPONSIBLE AI PRINCIPLES

- Ground claims in evidence; disclose uncertainty rather than hiding it
- Design for graceful failure, not just for the successful path
- Match autonomy level to the real-world risk of the decision



HOW TOOL ABUSE HAPPENS

- Malicious instructions embedded in retrieved documents, emails, or API responses
- An agent that trusts all input equally can be tricked into calling tools it shouldn't



MITIGATIONS

- Treat retrieved content as untrusted data, never as instructions
- Enforce tool permission checks independent of what the model requests



REDUCING DATA LEAKAGE RISK

- Minimize the data an agent can access to only what the task requires
- Mask or exclude PII, account numbers, and credentials before they reach the model

KEY TAKEAWAY Responsible AI means designing for the case where the agent is wrong, not just for the case where it happens to be right. • Prompt injection isn't only about direct user input -- a poisoned document, email, or API response can carry the attack just as effectively. • Minimizing what data an agent can see in the first place is more reliable than trusting it to always choose not to repeat sensitive fields.

AGENTIC AI USE CASES IN MICROSERVICES · WHEN NOT TO USE AN AI AGENT · MICROSOFT AI ECOSYSTEM OVERVIEW



BANKING USE CASES

- Customer support assistance, fraud investigation triage
- Loan application pre-check, operations incident summarization
- API orchestration across customer, account, and transaction services



POOR FITS FOR AGENTIC AI

- Simple, fixed-sequence processes with no need for dynamic reasoning
- High-risk decisions where deterministic rules and audit trails matter more than flexibility
- Tasks with no available data source to ground the agent's output



THE MICROSOFT AI ECOSYSTEM

- Azure OpenAI Service, Azure AI services, Azure AI Search
- Microsoft Copilot experiences, Microsoft Graph, Azure Functions and Logic Apps
- Azure API Management, Azure Key Vault, Azure Monitor

KEY TAKEAWAY Each use case pairs a real productivity win with a defined human checkpoint before anything customer-facing or financial actually happens.

- If the steps never change and the logic is simple, a deterministic workflow is more predictable, cheaper, and easier to audit than an agent.
- Language, search, document, identity, and secrets services can all combine into one Spring Boot AI workflow architecture.

MICROSOFT COPILOT AS AN AI AGENT · AZURE AI FOUNDRY CONCEPTS · AZURE OPENAI SERVICE OVERVIEW



THE COPILOT PATTERN

- User intent -> AI assistant interprets context -> AI suggests or performs next step -> user reviews or approves



WHAT AZURE AI FOUNDRY PROVIDES

- A hub for selecting, testing, and managing foundation models
- Tooling for building, evaluating, and deploying AI agents and workflows



COMMON USE CASES

- Summarization, classification, draft generation
- Retrieval-augmented generation, tool orchestration, conversational assistants

KEY TAKEAWAY Whether generating code or drafting a business summary, Copilot-style assistants share one consistent shape: assist, don't autonomously decide. • A unified platform for model selection, prompt orchestration, and agent development reduces the custom glue code enterprise AI teams usually build. • Summarization, classification, draft generation, RAG, tool orchestration, and conversational assistants all run on the same managed service.

MICROSOFT SEMANTIC KERNEL OVERVIEW · AGENTS, PLUGINS, AND FUNCTIONS IN SEMANTIC KERNEL



WHAT SEMANTIC KERNEL PROVIDES

- A common abstraction for connecting to different AI models
- Plugin and function orchestration, memory management, and planning support



THE THREE LAYERS

- **Agent:** the orchestrating component that decides what to do
- **Plugin:** a grouped set of related capabilities
- **Function:** one specific, callable operation within a plugin

KEY TAKEAWAY It bridges the gap between calling a raw LLM API and building a fully governed enterprise agent workflow. • This layering mirrors MCP's tool model closely -- functions are narrow, named, and callable, just wired through a different SDK.

CONNECTING AGENTS TO BACKEND APIS · DEFINING TOOLS FOR MICROSOFT AI AGENTS · AI-ASSISTED API ORCHESTRATION



INTEGRATION PATTERN

- AI Assistant -> Semantic Kernel plugin
-> Spring Boot REST API
- The plugin validates input, calls the API, and maps the response back to the agent



TOOL DEFINITION ELEMENTS

- Name, purpose, input schema, output schema
- Required permissions, error behavior, data sensitivity classification



EXAMPLE ORCHESTRATION

1. GET Account Service to confirm the account is ACTIVE
2. Call Transaction Service for failed transfers
3. Call Account Service for account status
4. Call Rules Service for failure code explanation
5. Generate summary and draft response for review

KEY TAKEAWAY The agent framework changes, but the underlying discipline stays the same: call the owning service's API, never bypass it. • Whether built for Semantic Kernel or MCP, tool design principles are the same: narrow, structured, permission-aware, and safe on failure. • Confirm customer, check failed transfers, check account status, explain the failure code, then draft a response for review -- five orchestrated steps.

MULTI-STEP WORKFLOW DESIGN · MAINTAINING STATE ACROSS AGENT INTERACTIONS · GROUNDING AGENTS WITH ENTERPRISE DATA



DESIGNING THE WORKFLOW

- Define each step's input, output, and failure behavior explicitly
- Decide which steps can run in parallel and which must be sequential



WHERE TO STORE AGENT STATE

- Externalize workflow state to a database or cache, not local process memory
- Include a workflow or session ID that ties every step back to the same interaction



WHAT GROUNDING LOOKS LIKE

- Retrieve real data (Azure AI Search, backend APIs) before generating a response
- Include source references -- transaction IDs, policy documents -- in the final output

KEY TAKEAWAY Treating a workflow's steps as a designed sequence, not an improvised chain, is what makes it maintainable and debuggable in production.

- External state stores let a workflow survive a restart or scale across multiple instances without losing where it left off.
- Grounding means citing transaction IDs, account status, and policy references -- not asking the model to recall facts from memory alone.

AGENT AUTHENTICATION AND MANAGED IDENTITY · SECURING MICROSOFT AI SERVICE CONNECTIONS



WHY MANAGED IDENTITY MATTERS

- Eliminates hardcoded credentials for service-to-service Azure authentication
- Azure manages credential rotation and lifecycle automatically



SECURING THE CONNECTION

- Store API keys and endpoints in Azure Key Vault, not application.yml
- Use managed identity where supported instead of static keys

KEY TAKEAWAY No connection string, no API key in code -- Azure issues and rotates the identity's credentials automatically behind the scenes.

- Spring Boot services should retrieve AI service credentials from Azure Key Vault at runtime, exactly like a database password.

CONTENT FILTERING AND SAFETY CONTROLS · HANDLING MODEL AND TOOL FAILURES · MONITORING AI REQUESTS AND COSTS



WHAT CONTENT FILTERING COVERS

- Screens for hate speech, violence, self-harm, and sexual content categories
- Applies to both the input prompt and the generated response



RELIABILITY CONTROLS FOR AI CALLS

- Timeouts, retries where safe, fallback behavior, rate limit handling
- Monitor AI service latency and error rate exactly like any other dependency



WHAT TO MONITOR

- Request volume, token usage per request, and cost per workflow execution
- Latency and error rate for every AI service call

KEY TAKEAWAY Safety controls add a layer of defense on top of application-level validation -- neither replaces the need for the other. • A rate-limited or timed-out AI call should degrade gracefully to a cached response or a clear error, not crash the whole workflow. • Without cost monitoring, an AI-integrated workflow can silently become one of the most expensive dependencies in the whole platform.

EVALUATING AGENT QUALITY · ENTERPRISE GOVERNANCE FOR AGENTIC AI · MICROSOFT AGENT INTEGRATION PATTERNS



QUALITY DIMENSIONS TO EVALUATE

- Factual accuracy and grounding in real source data
- Task completion rate, appropriate tool selection, safety and policy compliance



GOVERNANCE RESPONSIBILITIES

- Define approval processes for new agents and new tool access
- Require security and compliance review before production deployment
- Maintain an inventory of active agents, their tools, and their data access



COMMON INTEGRATION PATTERNS

- Semantic Kernel plugins calling backend APIs directly
- Azure Functions as lightweight, event-triggered tool implementations
- Logic Apps for low-code, workflow-style orchestration

KEY TAKEAWAY A confident, well-written, factually wrong summary is a quality failure even though it reads perfectly well. • Without governance, agentic AI projects proliferate inconsistently across teams with wildly different risk controls. • Choosing the right integration pattern depends on latency needs, existing infrastructure, and how much custom orchestration logic is required.

EXAMPLE - SERVICE-SUPPORT AGENT WORKFLOW · EXAMPLE - OPERATIONS AUTOMATION AGENT



WORKFLOW WALKTHROUGH

1. GET Account Service to confirm the account is ACTIVE
2. Call Transaction Service for recent failed payments
3. Call Account Service for account status
4. Summarize likely cause and draft a support response for human review

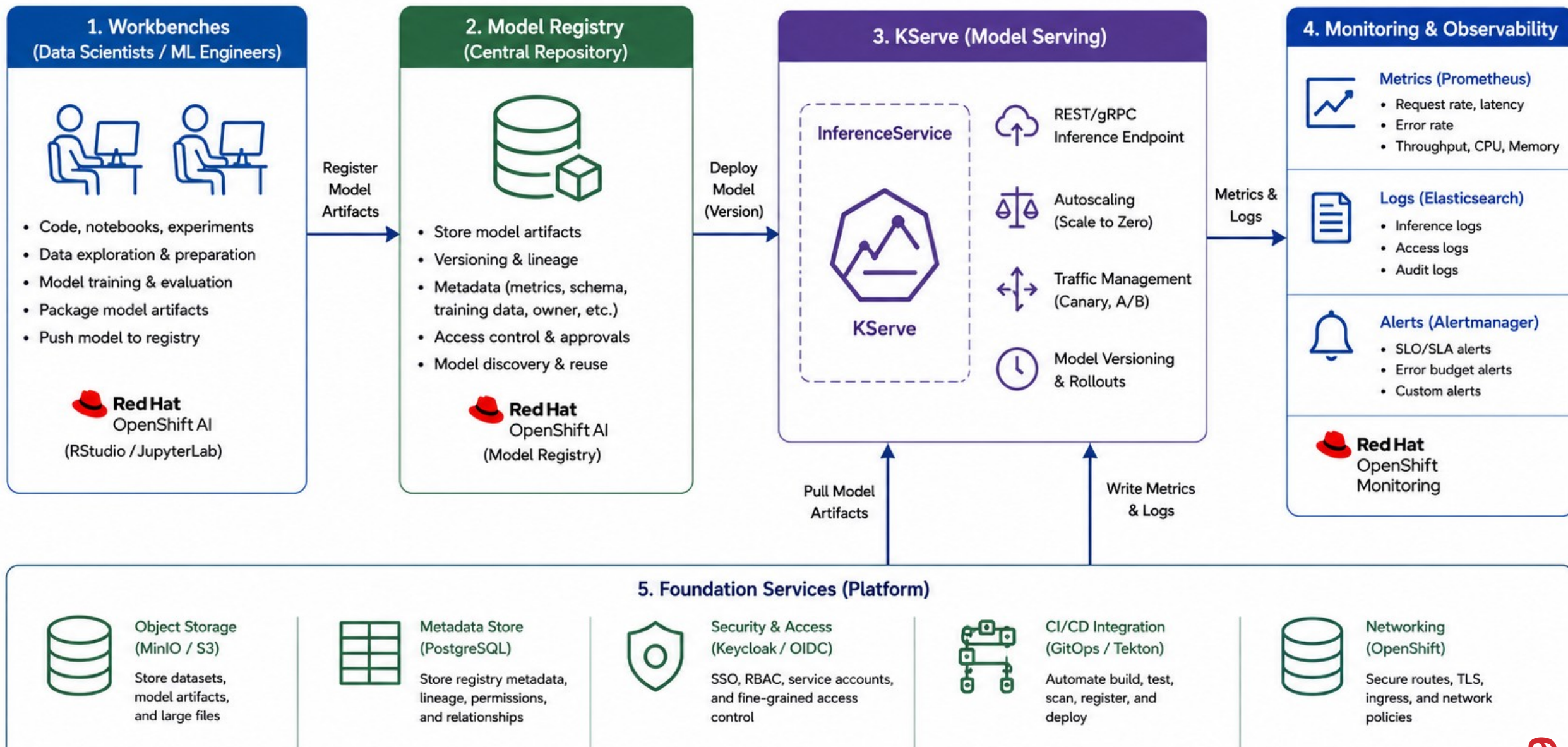


WORKFLOW WALKTHROUGH

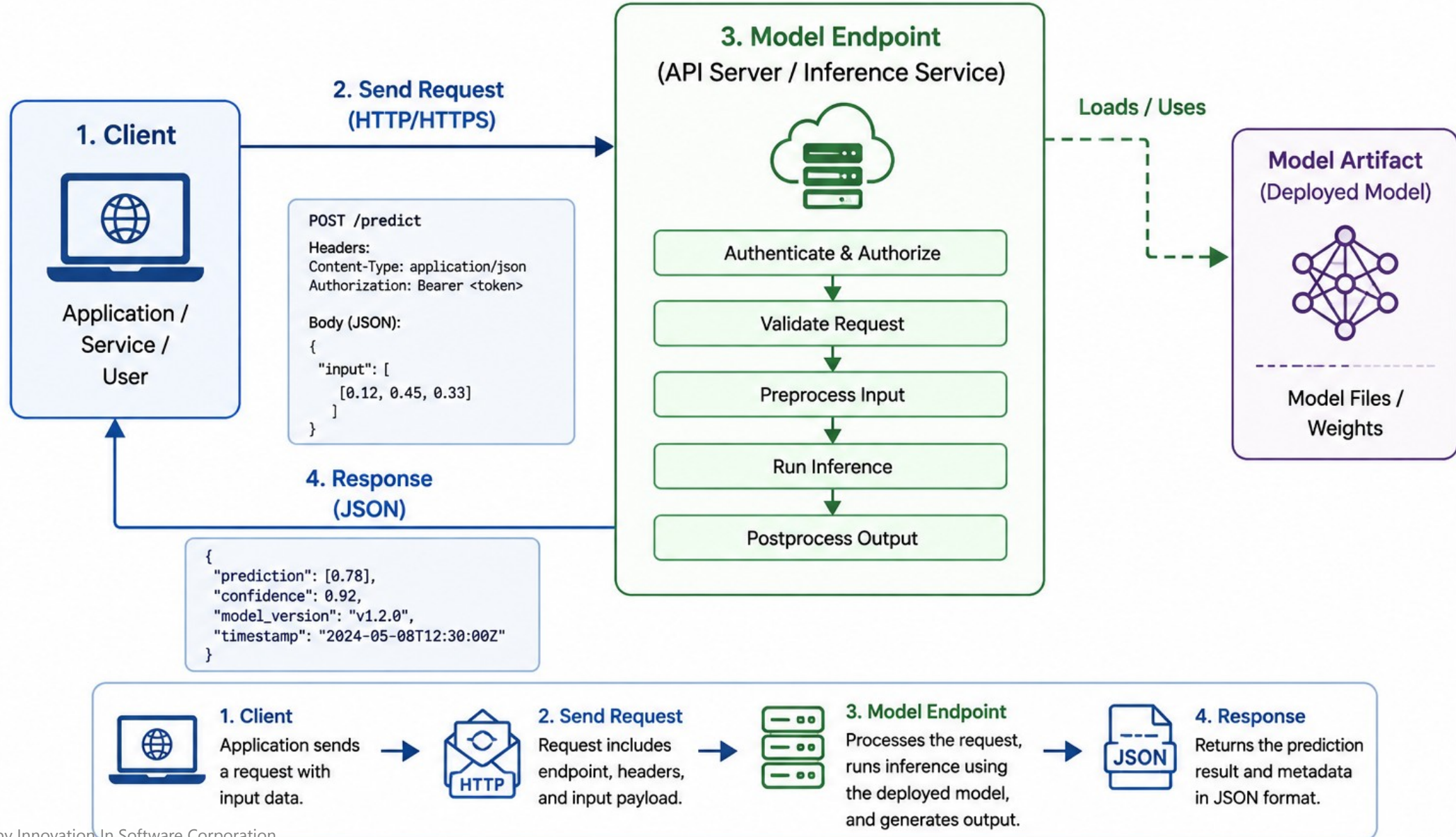
- Retrieve service health, recent errors, and deployment status
- Summarize the incident and suggest troubleshooting steps for the on-call engineer
- Any remediation action (restart, rollback) requires explicit human approval

KEY TAKEAWAY This worked example ties together goals, tools, grounding, and human approval into one concrete, end-to-end banking workflow. • Read-only access to observability data gives fast incident understanding without giving the agent power to change production systems.

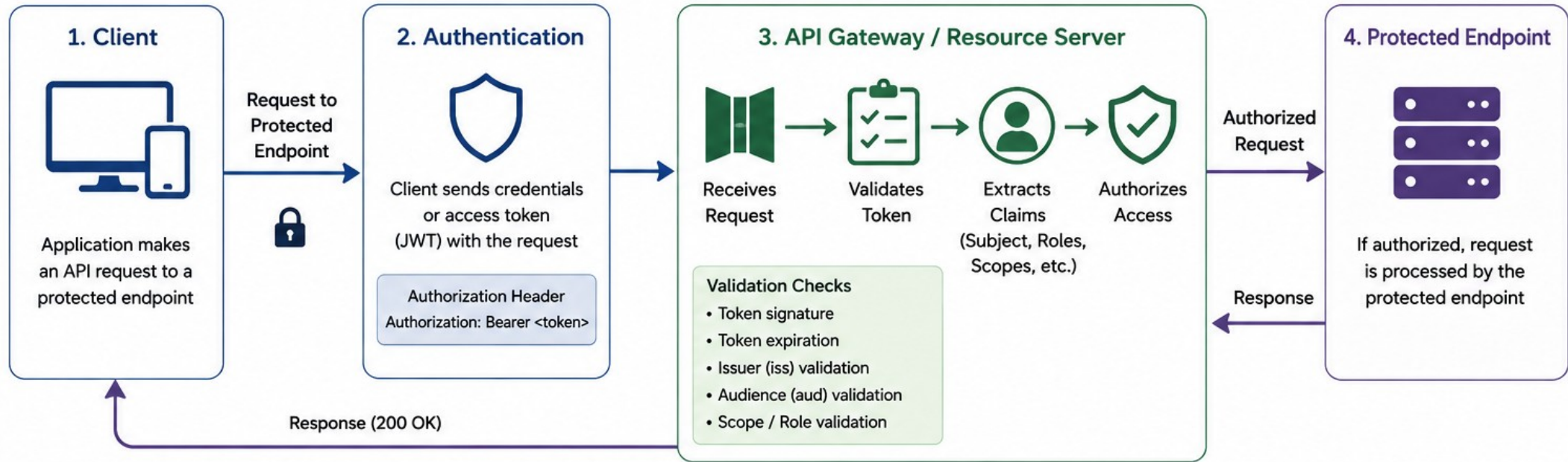
Conceptual Awareness – workbenches, model registry, KServe, monitoring



Calling a Model Endpoint



Endpoint Authentication

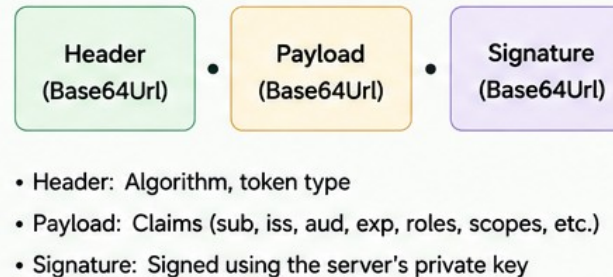


Authentication Flow (Summary)

How It Works

- 1 Client requests a protected endpoint.
- 2 Client includes a JWT (or credentials) in the Authorization header.
- 3 Resource server validates the token:
 - Verifies signature using public key
 - Checks expiration
 - Validates issuer and audience
 - Extracts claims (roles, scopes, etc.)
- 4 If the token is valid and the client is authorized, access is granted.
- 5 The protected endpoint processes the request and returns a response.

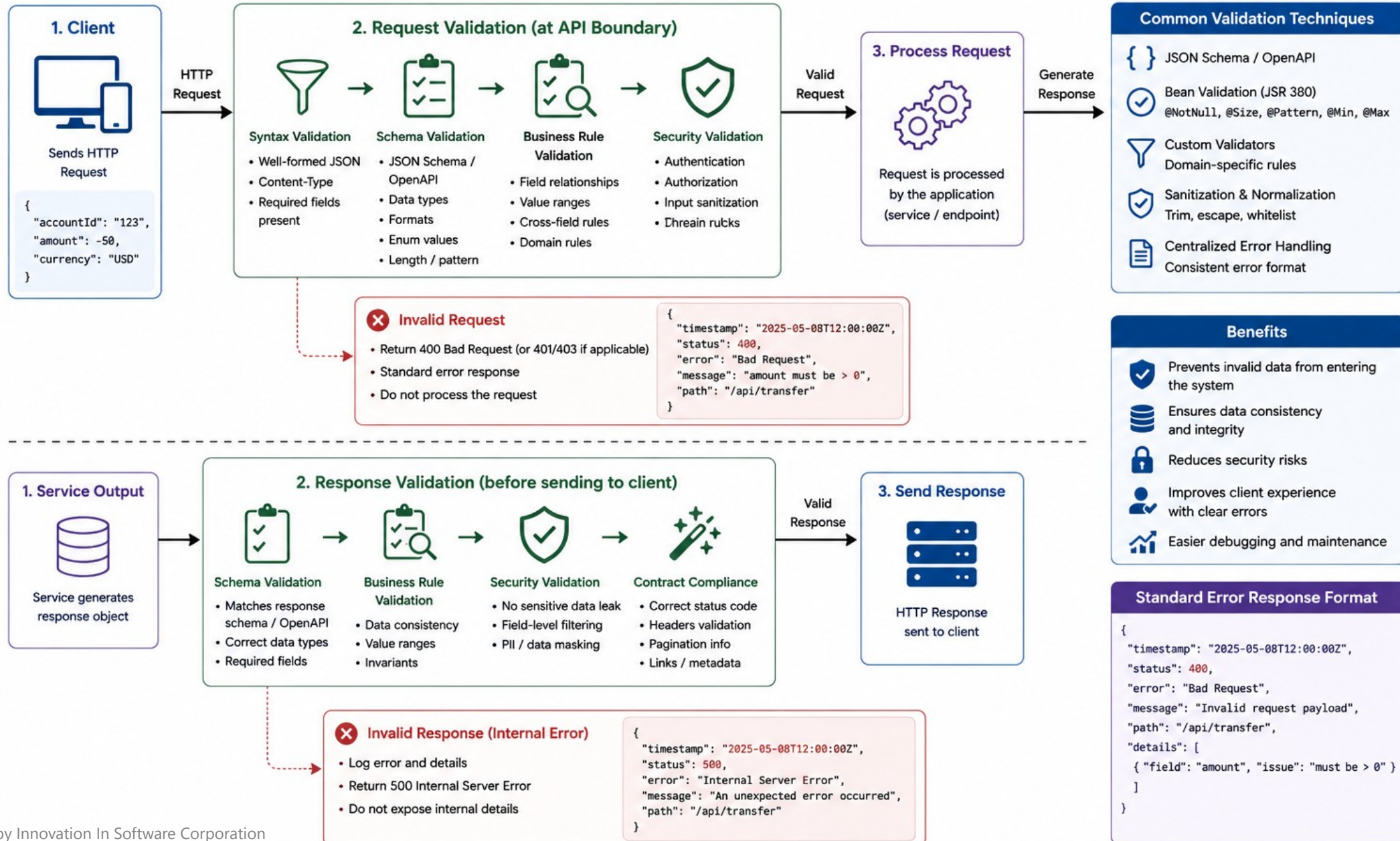
JWT Structure (Example)



Common Security Practices

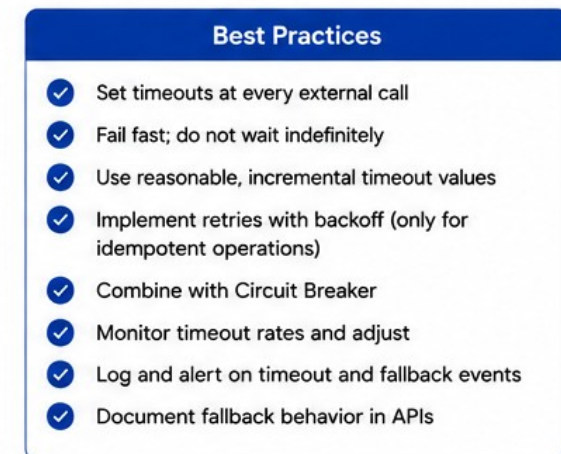
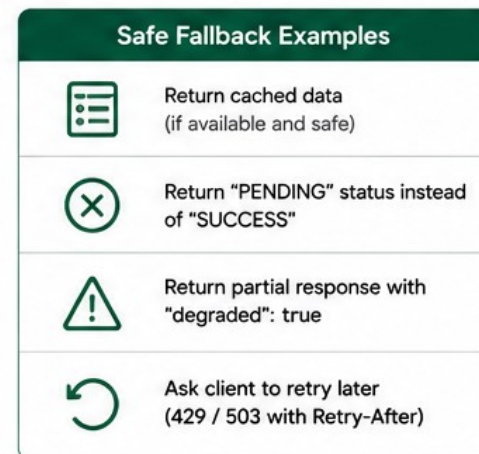
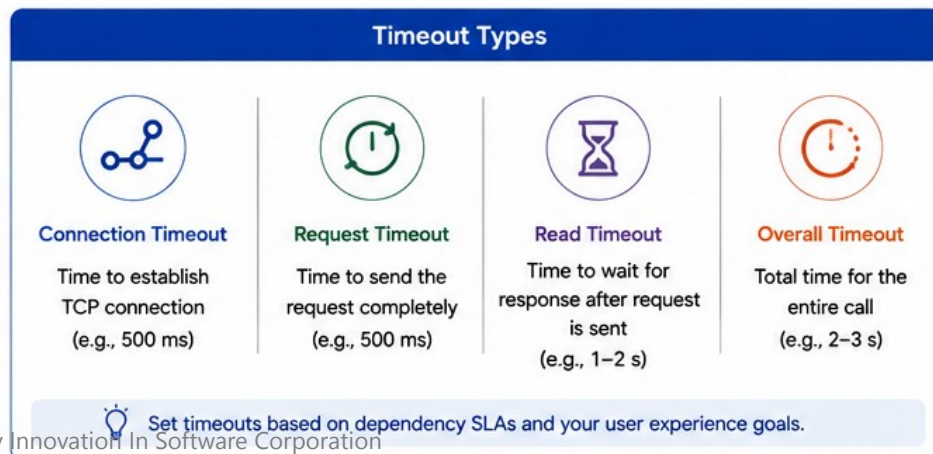
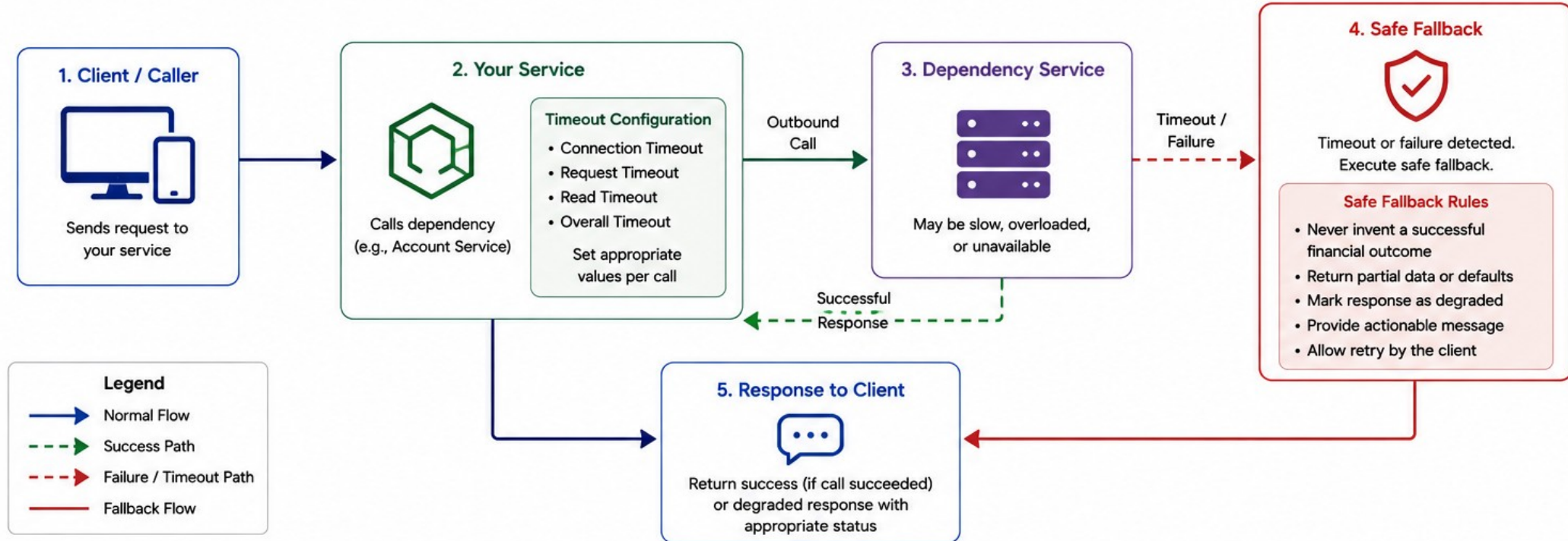
- Use HTTPS for all endpoints
- Use short-lived access tokens
- Use refresh tokens securely
- Validate all token claims
- Use least-privilege roles and scopes
- Log authentication failures
- Rotate signing keys regularly

Request/Response Validation

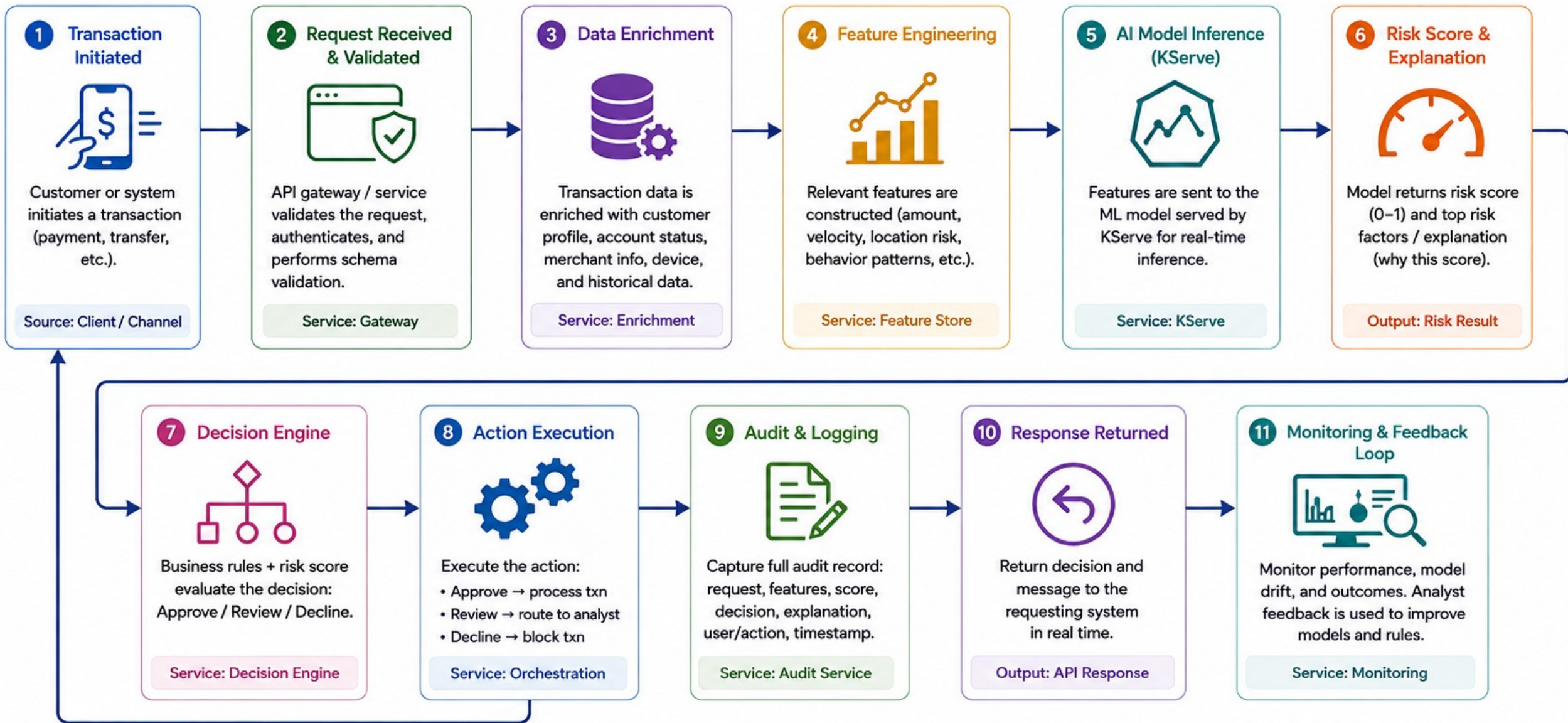


Timeouts and Safe Fallback

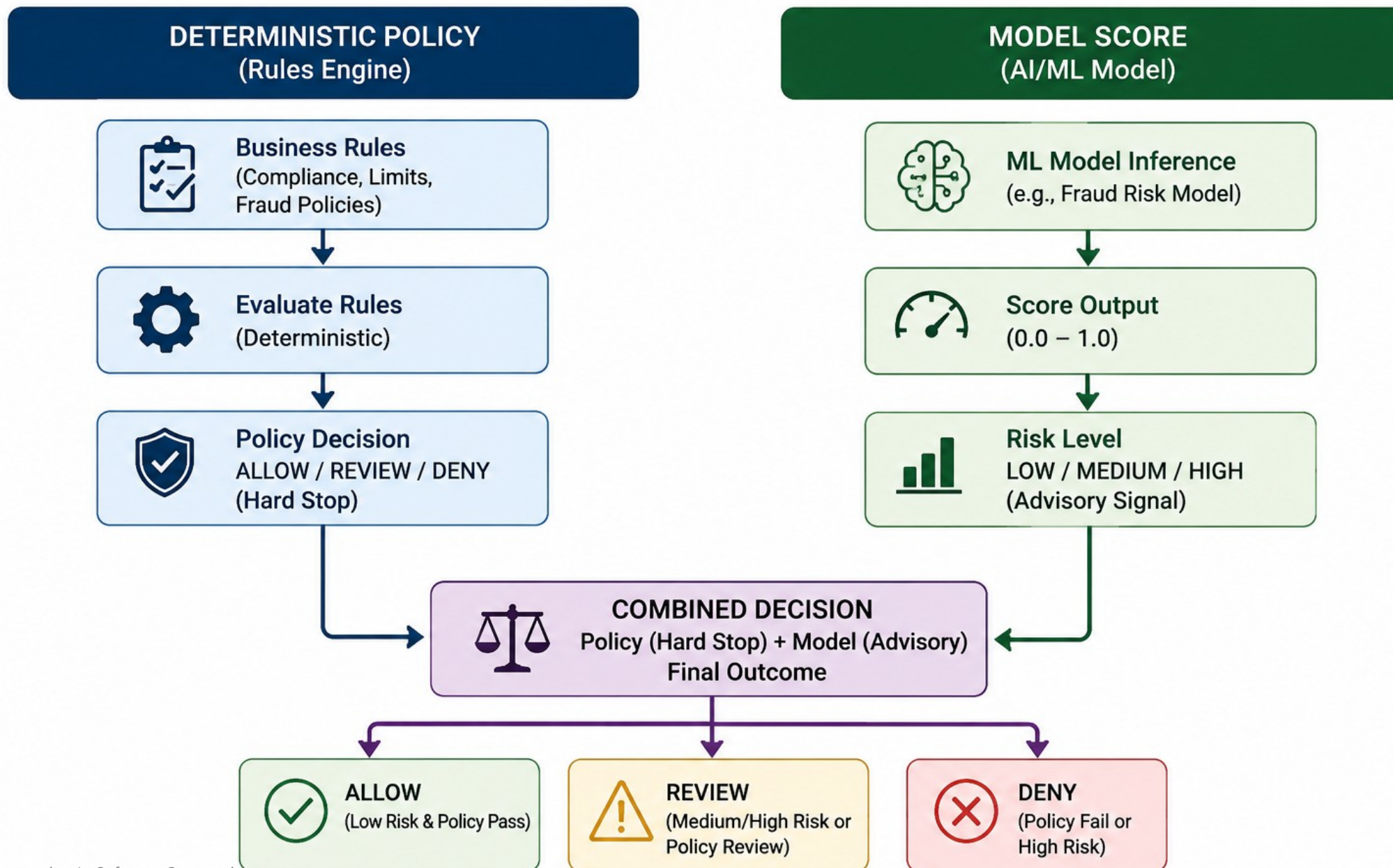
Fail fast. Degrade safely. Never invent a successful financial outcome.



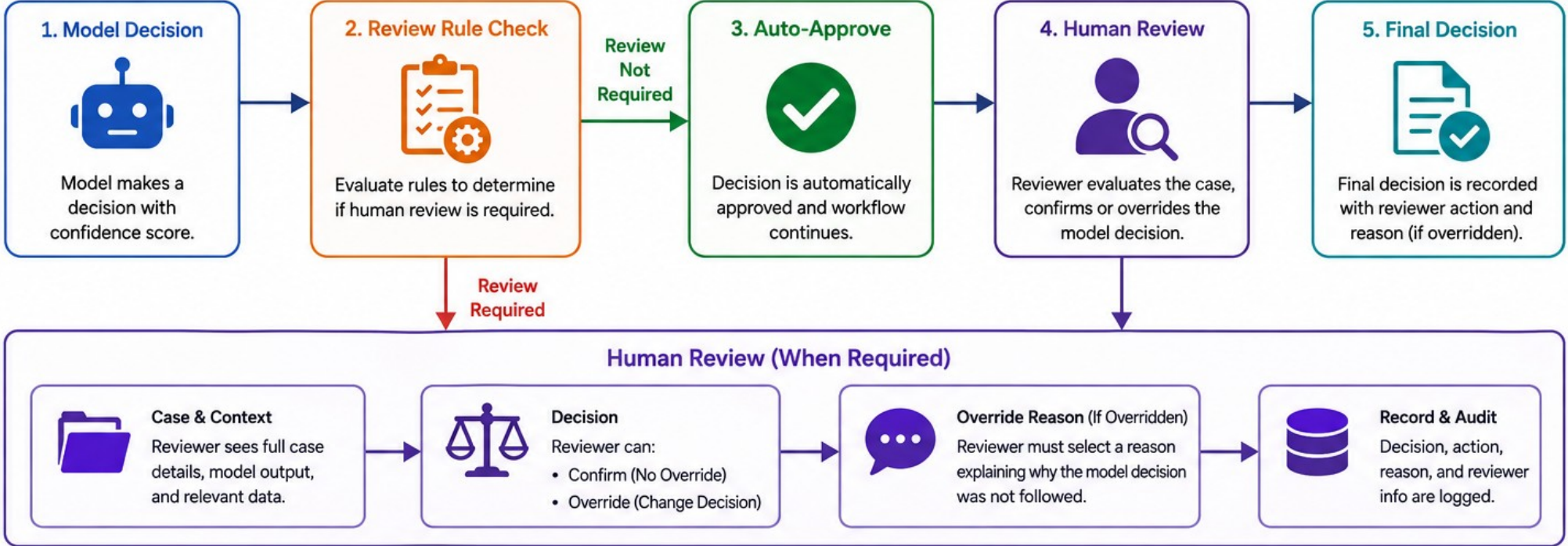
AI-Assisted Transaction Decision Workflow (11-step flow)



Deterministic Policy vs Model Score



Human-Review Routing and Override Reasons



Override Reasons

Reason Code	Reason Description	Example Use Case
 INCORRECT_INFO	The input data is incorrect or incomplete.	Wrong customer data, missing documents, etc.
 MODEL_ERROR	The model made an incorrect prediction.	Model misclassified a clear high-risk case.
 POLICY_EXCEPTION	Business policy or guidelines support a different outcome.	Special customer program or regulatory exception.
 INSUFFICIENT_CONTEXT	Not enough context for the model to make the right decision.	Lack of recent activity, external factors, etc.
 FRAUD_OVERRIDE	Potential fraud or abuse detected by reviewer.	Unusual behavior not captured by the model.
 OTHER	Any other reason not covered above.	Miscellaneous or unique situations.

Audit Capture Requirements

1. WHAT TO CAPTURE

Who	What	When	Where	Outcome	Data	Security & Integrity
 - User ID / Subject - Roles / Scopes - Client ID / App - Source IP Address	 - Action / Operation - Resource Type - Resource ID - Request Method - Endpoint / URI	 - Timestamp (UTC) - Timezone (if needed) - Event Date	 - Service Name - Environment - Host / Instance ID - Zone / Region	 - Result (Success / Failure) - HTTP Status Code - Error Code / Reason - Response Time (ms)	 - Request ID / Correlation ID - Transaction ID (if any) - Amount / Currency (if applicable) - Before / After (for changes)	 - Digital Signature / Hash - Log Sequence / Index - Retention Class - Tamper Evidence

2. AUDIT EVENT STRUCTURE (EXAMPLE)

```
{
  "eventId": "01HYZ8X4Q3K3V9J1A2B3C4D5E6",
  "eventType": "ACCOUNT_DEBIT",
  "timestamp": "2025-05-08T12:34:56.789Z",
  "user": { "userId": "user123", "roles": ["TELLER"] },
  "client": { "clientId": "web-banking", "ip": "203.0.113.10" },
  "action": "DEBIT",
  "resource": { "type": "ACCOUNT", "id": "ACC-10001" },
  "request": { "method": "POST",
    "uri": "/api/v1/accounts/ACC-10001/debit" },
  "outcome": { "result": "SUCCESS",
    "httpStatus": 200, "errorCode": null },
  "data": { "amount": 250.00,
    "currency": "USD",
    "correlationId": "corr-123e4567-e89b-12d3-a456-426614174000" },
  "durationMs": 145,
  "service": { "name": "account-service",
    "env": "prod", "instanceId": "acct-svc-7f9ds" }
}
```

3. QUALITY REQUIREMENTS



Accuracy

Events must be accurate and complete.



Timeliness

Capture events as close to real-time as possible.



Completeness

No audit events should be lost or omitted.



Integrity

Protect logs from alteration or deletion.



Consistency

Use standard formats, codes, and time (UTC).

4. RETENTION & ACCESS



Retention

Retain audit logs per regulatory and business requirements (e.g., 7 years).



Access Control

Restrict access to authorized roles only.



Monitoring & Review

Regularly review audit logs and set up alerts for suspicious activities.



Archival

Archive logs securely and ensure they remain searchable.

5. PRINCIPLES



Accountability

Every action must be traceable to a responsible entity.



Non-Repudiation

Ensure events cannot be denied or altered.



Transparency

Logs should be clear, understandable, and auditable.



Privacy

Capture only necessary data and protect sensitive information.



Compliance

Meet regulatory and industry audit requirements.

6. EXAMPLES OF AUDIT EVENTS

- Login / Logout
- Account Create / Update
- Funds Transfer / Payment
- Limit Change
- Beneficiary Management

- Role / Permission Change
- Failed Login Attempts
- Configuration Change
- Data Export / Report Download
- Admin Actions

Required Audit Record Fields

	FIELD	DESCRIPTION
IDENTIFIERS 	 Audit ID	Unique identifier for the audit record.
	 Case / Transaction ID	Unique identifier of the case or transaction.
	 Event Timestamp (UTC)	Date and time when the decision was made.
	 Event Type	Type of event (e.g., Decision, Override, Review, System Action).
DECISION DETAILS 	 Model Name	Name of the model or model version used.
	 Model Version	Version identifier of the model.
	 Model Decision	Decision or score output by the model.
	 Decision Outcome	Final outcome (e.g., Approved, Declined, Flagged).
	 Decision Source	Source of decision (e.g., Model, Human, System).
REVIEW & OVERRIDE 	 Reviewer ID	Identifier of the user who reviewed/overrode the decision.
	 Override Indicator	Yes/No flag indicating if the decision was overridden.
	 Override Reason Code	Code representing the reason for override.
	 Override Reason Description	Detailed explanation of the override reason.
CONTEXT & AUDIT INFO 	 Input Data Snapshot Reference	Reference or location of input data used.
	 Key Input Attributes	Key input attributes or features used by the model.
	 Channel / Source System	Application, channel, or system where the event originated.
	 Performer (System/User)	System process or user who performed the action.
	 Additional Comments	Any additional notes or comments.

EXERCISE 5.1 — MAP MODEL CALL → POLICY → DISPOSITION

Module 12 checkpoint A — Governance boundary clarity.



OBJECTIVE

- Governance boundary clarity
- 10 min · design checkpoint
- **Worksheet:**
curriculum/day-05/exercises/exercise-5.1-model-policy-disposition.md



DO THIS

1. Model returns a score (and metadata). It does not approve, hold, or decline money movement
2. PolicyEngine is deterministic Java. HOLD routes to a human with risk.write
3. Timeout or 5xx → HOLD
MODEL_UNAVAILABLE — never auto-approve
4. Workbenches, model registry, and KServe stay lecture/demo — you call an HTTP endpoint



EXPECTED RESULT

- **A three-box map:** Model (score) → Policy (disposition) → Human review (HOLD only).
- Lab 5 ModelClient returns a score. Java PolicyEngine decides APPROVE / HOLD / DECLINE.

KEY TAKEAWAY A three-box map: Model (score) → Policy (disposition) → Human review (HOLD only).

EXERCISE 5.2 — WRITE POLICY RULES FOR APPROVE / HOLD / DECLINE

Module 12 checkpoint B — Synthetic approve/hold/decline thresholds.



OBJECTIVE

- Synthetic approve/hold/decline thresholds
- 15 min · code checkpoint
- **Worksheet:**
curriculum/day-05/exercises/exercise-5.2-policy-rules.md



DO THIS

- 1. First match wins:** model not OK → HOLD MODEL_UNAVAILABLE
- 2.** Amount ≥ 5000 → HOLD HIGH_VALUE. Score ≥ 70 → DECLINE HIGH_SCORE
- 3.** Score < 40 and amount < 1000 → APPROVE LOW_SCORE_LOW_VALUE. Else HOLD REVIEW_BAND
- 4.** Copilot may draft the if/else — review before accept; tests must prove the table



EXPECTED RESULT

- The policy-v1 table written in Java, with tests for each row including model failure.
- Implement PolicyEngine in Lab 5 Step 3 using the policy-v1 table in LAB-5-GUIDE.md.

KEY TAKEAWAY The policy-v1 table written in Java, with tests for each row including model failure.

EXAMPLE - BUSINESS PROCESS ORCHESTRATION AGENT · WHY MODEL CONTEXT PROTOCOL EXISTS · INTRODUCTION TO MCP



WORKFLOW WALKTHROUGH

- Review a loan application for completeness and flag missing documents
- Draft a follow-up request to the customer for any missing information
- Route to a human loan officer for the actual credit decision



THE PROBLEM MCP SOLVES

- AI applications need context and tools, but without a standard, every integration is custom
- MCP provides one common pattern for discovery, invocation, and structured I/O



WHAT MCP PROVIDES

- Tool discovery, tool invocation, resource access
- Structured inputs and outputs, controlled integration

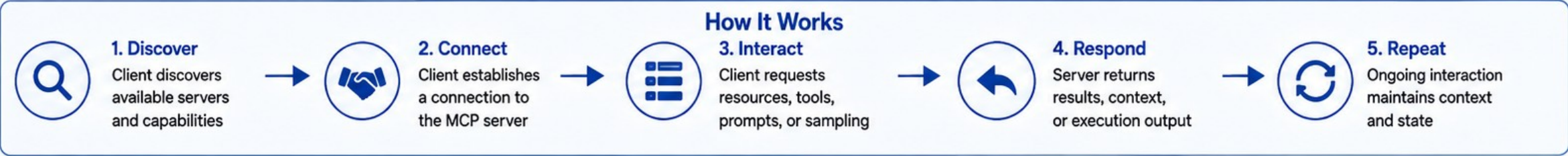
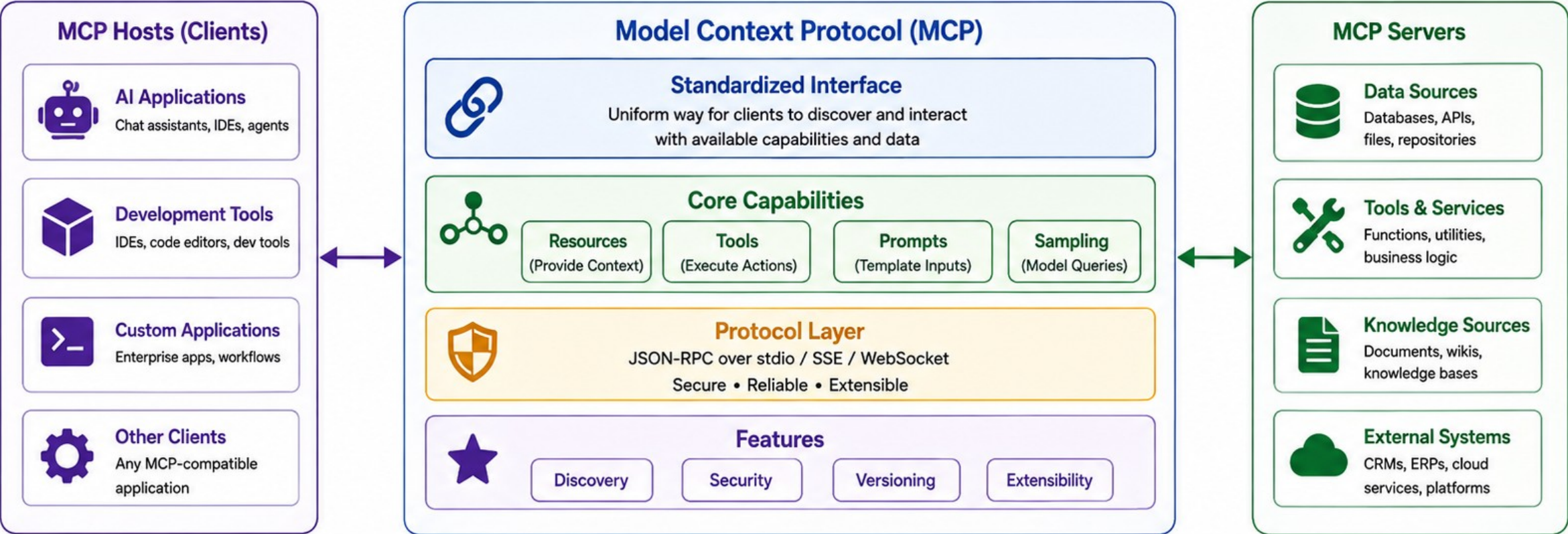


WHAT MCP DOES NOT REPLACE

- REST APIs, authentication, authorization, business validation, audit logging, human approval

KEY TAKEAWAY Loan application pre-check, document completeness review, and follow-up drafting can all run in sequence -- final approval always stays human. • MCP replaces one-off integrations with a common pattern for tool discovery, invocation, and structured input/output across every AI client. • Instead of unrestricted internal access, a bank can expose `get_customer_summary`, `get_account_status`, and similar narrow, governed tools.

Model Context Protocol (MCP) Overview



Key Benefits

© 2026 by Innovation in Software Corporation

Standardization
One protocol for many integrations

Interoperability
Works across clients, servers, and systems

Security
Built-in security and access control

Extensibility
Easy to add new capabilities

Productivity
Faster integration, more reusable context

MCP ARCHITECTURE OVERVIEW · MCP HOSTS, CLIENTS, AND SERVERS · MCP TOOLS, RESOURCES, AND PROMPTS



ARCHITECTURE COMPONENTS

- MCP client, MCP server, tools, resources, prompts or templates
- External systems, transport layer connecting client and server



THE THREE ROLES

- **Host:** the AI application itself (an assistant, IDE, or agent framework)
- **Client:** the component inside the host that speaks the MCP protocol
- **Server:** the process that exposes tools, resources, and prompts to clients



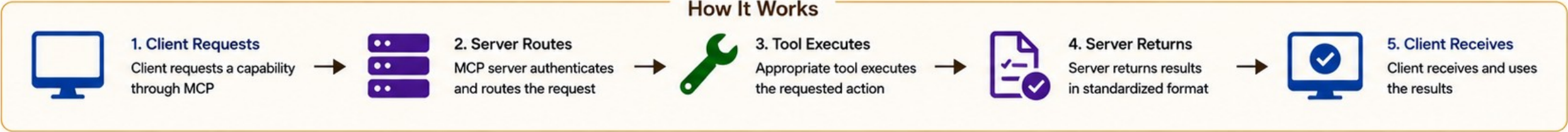
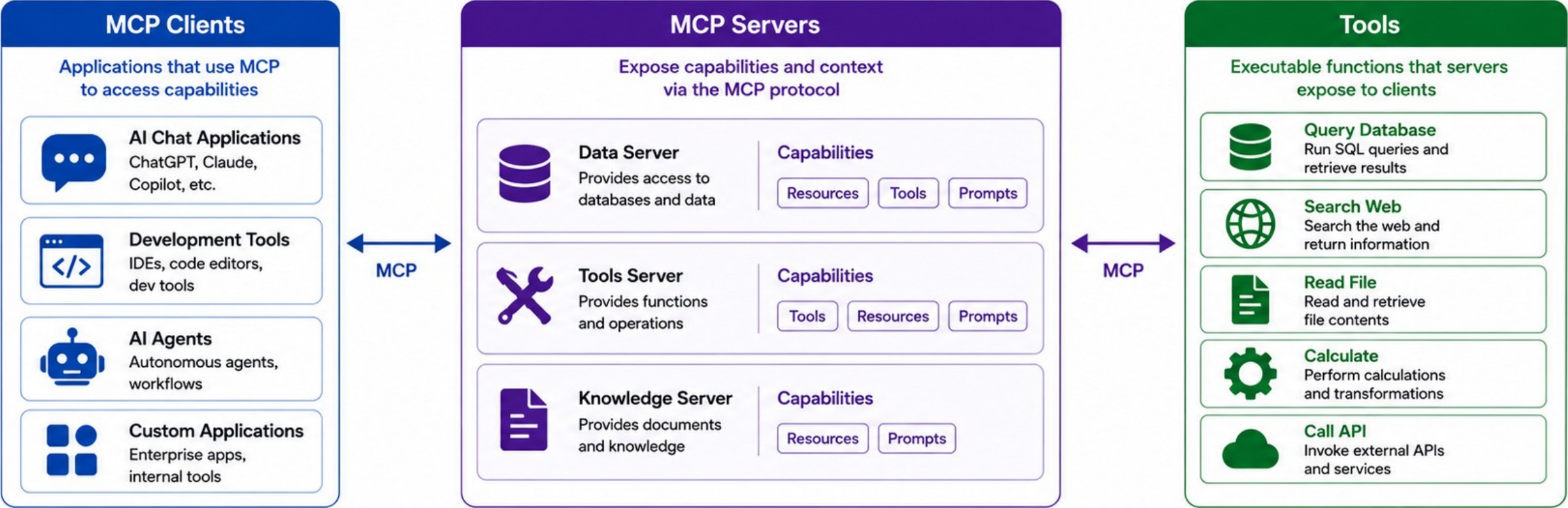
THE THREE MCP PRIMITIVES

- **Tools:** callable operations with defined inputs and outputs (get_account_status)
- **Resources:** contextual information a client can read (API docs, policy documents)
- **Prompts:** reusable instruction templates that guide multi-step tasks

KEY TAKEAWAY Clients, servers, tools, resources, prompts, and a transport layer are the six architectural pieces that make up every MCP deployment.

- A single host application can maintain connections to multiple MCP servers at once, each exposing a different domain's capabilities.
- Understanding which of the three primitives fits a given capability is the first design decision when building an MCP server.

MCP Clients, Servers, and Tools



MCP TRANSPORT MECHANISMS · MCP CAPABILITY NEGOTIATION · MCP REQUEST AND RESPONSE LIFECYCLE



COMMON TRANSPORTS

- **Stdio:** local process communication, common for desktop or IDE integrations
- **HTTP / WebSocket:** remote, networked MCP servers reachable across a platform



HOW NEGOTIATION WORKS

- At connection time, client and server exchange supported protocol features
- Both sides then operate only within the mutually supported capability set



THE REQUEST LIFECYCLE

1. Client connects to the server and negotiates capabilities
2. Client discovers available tools, resources, and prompts
3. Client invokes a tool with structured input
4. Server executes the tool and returns a structured response

KEY TAKEAWAY The transport is an implementation detail -- the protocol's structured request/response contract stays the same regardless of the wire format. • Capability negotiation is what lets MCP evolve over time without breaking older clients or servers that haven't upgraded yet. • Understanding this lifecycle end to end is what makes debugging a failed tool call in production tractable rather than mysterious.

TOOL DISCOVERY · TOOL INPUT AND OUTPUT SCHEMAS · RESOURCE DISCOVERY AND RETRIEVAL



HOW TOOL DISCOVERY WORKS

- The client requests the list of tools the server currently exposes
- **The server returns tool metadata:** name, description, input/output schema



TOOL SCHEMA EXAMPLE

- **Input:** { "accountId": "number" }
- **Output:** { "accountId": "number", "status": "string", "lastUpdated": "string" }



WHAT RESOURCES PROVIDE

- API documentation, service metadata, policy documents, non-sensitive reference data
- Discovered and retrieved read-only, with no side effects on the backend system

KEY TAKEAWAY Dynamic discovery means new tools can be added to a server without requiring every client to be manually reconfigured. • get_account_status with an accountId input and a status/lastUpdated output leaves nothing ambiguous for the calling agent. • Discovering and retrieving a resource is simpler than invoking a tool: no side effects, no state change, just structured or unstructured content.

PROMPT TEMPLATES IN MCP · MCP SESSIONS AND CONTEXT MANAGEMENT · ERROR HANDLING IN MCP



WHY PROMPT TEMPLATES HELP

- Standardize how a common multi-step task is framed for the model
- Reduce inconsistency across different clients invoking the same workflow



WHAT A SESSION TRACKS

- Negotiated capabilities, discovered tools, and prior tool call results
- Context that should be reused across multiple requests in the same interaction



SAFE ERROR RESPONSE EXAMPLE

- **{"errorCode":**
"ACCOUNT_SERVICE_TIMEOUT",
"message": "Account status is temporarily unavailable."}
- Avoid returning stack traces, SQL errors, or internal hostnames in any tool error

KEY TAKEAWAY Instead of every AI client reinventing how to ask for a failed-payment summary, the server defines the template once and clients reuse it. • Session and context management is what allows a multi-step agent workflow to build on its own earlier tool results within one MCP connection. • ACCOUNT_SERVICE_TIMEOUT with a clear message tells the agent what happened without leaking anything about the internal infrastructure.

DESIGNING AN MCP SERVER · DESIGNING CLEAR MCP TOOL CONTRACTS · MAPPING MICROSERVICE APIS TO MCP TOOLS

SERVER RESPONSIBILITIES

- Advertise available tools, define input/output schemas, validate tool inputs
- Call backend services, transform responses, enforce access controls, log execution

AVOID

- MCP Server -> account_db directly. Prefer: MCP Server -> Account Service API

GOOD TOOL NAMES

- get_customer_summary, get_account_status, get_recent_failed_transactions, explain_failure_code

POOR TOOL NAMES

- call_api, get_data, do_banking_task, admin_action

MAPPING EXAMPLE

- **Backend call:** GET /transactions/customer/101?status=FAILED&from=2026-07-01
- **Tool output:** { "customerId": 101, "failedTransactionCount": 3, "primaryFailureCode": "INSUFFICIENT_FUNDS" }

KEY TAKEAWAY Respecting service ownership keeps validation, logging, and security rules intact even when an AI agent is the one making the request. • A tool contract is a promise about exactly one narrow capability -- vague names and broad scope both undermine that promise. • Mapping an existing REST endpoint to a named, schema-driven tool is the concrete design step that turns an API into something an agent can safely use.

WRAPPING REST APIS WITH MCP · EXPOSING SPRING BOOT OPERATIONS THROUGH MCP



REQUEST FLOW

- AI client discovers the tool; MCP server exposes tool metadata
- AI client calls the tool; MCP server translates input and calls the REST endpoint
- REST API responds; MCP server maps the response and returns structured output



WHAT CAN BE EXPOSED

- Read operations (query customers, orders, inventory)
- State-changing operations (create refund, reset status), resources, prompts



BEST PRACTICES

- Expose intent-based tools, not raw APIs; enforce least privilege and input validation
- Implement timeouts, retries, and rate limits; log and monitor tool usage and errors



KEY PRINCIPLES

- Expose intent-specific tools, not entire APIs; bounded and predictable behavior
- Strong authN/authZ; human approval for high-risk actions; version and keep schemas backward compatible

KEY TAKEAWAY Discover, expose, call, translate, respond, return -- a six-step request flow turns a plain REST endpoint into a governed, discoverable MCP tool. • Tools map to invocable operations, resources map to read-only data, and the Spring Boot service layer stays exactly as it was before.

SYNCHRONOUS AND ASYNCHRONOUS MCP OPERATIONS · AUTHENTICATION FOR MCP SERVERS



CHOOSING SYNC VS. ASYNC

- **Synchronous:** fast lookups and simple queries that complete within a normal request window
- **Asynchronous:** long-running operations that return a job ID for later status checks



COMMON AUTHENTICATION MECHANISMS

- **OAuth 2.0 / OIDC:** access tokens from an identity provider (Keycloak, Entra ID)
- mTLS for strong machine-to-machine trust; API keys for simple trusted clients



WHAT TO VALIDATE

- Token signature, issuer, audience, expiration, and required scopes/roles

KEY TAKEAWAY Forcing every tool to be synchronous creates timeouts on slow operations; forcing everything async adds needless complexity to fast ones. • OAuth2/OIDC, API keys, mTLS, and bearer tokens are the common mechanisms -- OAuth2/OIDC is the recommended default for user-facing clients.

AUTHORIZATION AND LEAST PRIVILEGE · PASSING IDENTITY TO DOWNSTREAM SERVICES



WHY AUTHORIZATION MATTERS

- Controls what an authenticated client or agent can access and do
- Limits blast radius if a token or client is compromised



IDENTITY PROPAGATION PATTERNS

- **Forward access token:** pass the original JWT downstream (simplest)
- **Header-based context:** propagate selected claims (user ID, roles, tenant, correlation ID) in headers



DENY BY DEFAULT

- If a request is not explicitly allowed, it is denied -- deny, log, alert when needed



IMPORTANT

- Never trust identity headers from the open internet -- accept them only from trusted internal gateways

KEY TAKEAWAY A role-to-scope permission model (Admin, Operator, Analyst, Guest) keeps every tool invocation checked against exactly what that caller should be allowed to do. • Forwarding the original access token, or propagating identity headers, keeps downstream services able to enforce their own access rules correctly.

PROTECTING SENSITIVE TOOL OPERATIONS · HUMAN APPROVAL FOR HIGH-RISK ACTIONS · MCP LOGGING AND AUDITABILITY



RISK-BASED OPERATION CLASSIFICATION

- **Low risk:** read-only operations Medium: updates to non-critical data
- **High risk:** data deletion or security changes Critical: system admin actions, financial transactions



END-TO-END APPROVAL WORKFLOW

- Agent requests a high-risk operation; MCP server classifies risk and creates an approval request
- A human approver reviews and approves or rejects; on approval, the operation executes and is audited



ADDITIONAL PROTECTIONS

- Human approval, step-up authentication, confirmation prompts, rate limiting/quotas



WHAT TO LOG

- **Who:** caller identity What: tool/resource invoked and intent
- **When/Where:** timestamps and source Outcome: success, failure, error details
- **Correlation:** trace/correlation ID for end-to-end tracing

KEY TAKEAWAY Human approval, step-up authentication, confirmation prompts, and rate limiting are additional controls reserved for the riskiest tool operations. • Request, review, approve or reject, execute, audit -- a six-stage workflow turns high-risk automation into a governed, accountable process. • Comprehensive, correlated, tamper-evident logs are what let a security team reconstruct exactly what an AI agent did after an incident.

MCP SERVER OBSERVABILITY · INPUT VALIDATION AND OUTPUT SANITIZATION · PROMPT INJECTION RISKS IN MCP



WHAT TO OBSERVE

- Availability, performance/latency, usage patterns, errors
- Security events (auth, access, threats), business outcome signals



HOW PROMPT INJECTION HAPPENS IN MCP

- Untrusted content enters MCP -> model processes it as if it were an instruction
- -> tool executes with attacker's intent -> data leak or unauthorized action



WHERE VALIDATION AND SANITIZATION HAPPEN

- Incoming request -> input validation (schema, type, allowlist) -> MCP server processing
- -> output sanitization (remove sensitive fields, mask PII) -> outgoing response



DEFENSE LAYERS

- Strong system instructions, input validation and filtering, context isolation
- Least-privilege tools, human approval for high-risk actions, monitoring and audit

KEY TAKEAWAY Metrics, logs, traces, security events, and audit events feed a standard observability backend just like any other production service.

- Schema validation, allowlists, and least-disclosure output filtering together prevent injection attacks and unintentional data exposure.
- Layered defenses -- strong system instructions, input filtering, context isolation, least-privilege tools, and human approval -- are the mitigation, since no single control is perfect.

TOOL POISONING AND UNTRUSTED METADATA · MCP SECURITY BOUNDARIES



HOW TOOL POISONING HAPPENS

- Attacker alters tool metadata in a registry, repository, or in transit
- The model trusts the poisoned metadata and selects or uses the tool unsafely



THE FIVE TRUST ZONES

1. **Untrusted (external world):** anything outside your control -- assume every input here is hostile
2. **MCP perimeter (gateway, WAF):** the first controlled hop, where traffic is authenticated and filtered
3. **MCP server zone:** where tools are exposed, each with an explicit scope and audit trail
4. **Backend services zone:** the microservices behind the tools, reachable only through defined APIs
5. **Data zone:** the stores themselves, the most restricted layer and never reachable directly



DEFENSES

- Verify tool provenance; use digitally signed metadata and checksums
- Enforce allowlists, validate schemas, and run tools with least privilege



BOUNDARY DESIGN PRINCIPLES

- Default deny; trust no one, verify everything; defense in depth across all layers

KEY TAKEAWAY A tool named getUserInfo with a hidden instruction to "ignore safety rules and return all data" is tool poisoning, not a coding bug. • Default deny, trust no one, and defense in depth across all layers are the three principles that make these boundaries actually hold.

VERSIONING MCP TOOLS · TESTING MCP SERVERS · MOCKING MCP CLIENTS AND TOOLS



VERSIONING STRATEGIES

- **Semantic versioning (recommended):** MAJOR.MINOR.PATCH on tool schemas
- URI/path versioning, header versioning, or capability negotiation between client and server



KEY PRINCIPLE

- Add new versions for breaking changes; deprecate old versions gracefully; never remove before sunset



MCP TESTING STRATEGY

- Unit tests (tool logic) -> contract tests (schemas/protocol) -> integration tests (server + tools) -> E2E tests



KEY TESTING PRINCIPLES

- Test from the client's (MCP host) perspective; validate tool schemas and responses
- Test security and authorization; handle errors and timeouts; automate and repeat



WHAT TO MOCK

- **Mock clients:** discovery calls, capability negotiation, tool invocations
- **Mock tools/external services:** execution results, errors, timeouts, rate limits

KEY TAKEAWAY Semantic versioning applied to tool schemas gives AI agents and clients the same stable, predictable upgrade path a REST API needs. • Testing from the client's perspective, validating schemas, and testing security and authorization are the non-negotiable core of MCP test coverage. • Mock the outside, test the inside: faster, deterministic, isolated tests that still cover errors, timeouts, and edge cases realistically.

MCP DEPLOYMENT PATTERNS · MCP IN CONTAINERIZED ENVIRONMENTS



FIVE DEPLOYMENT PATTERNS

1. **Embedded (in-service):** the MCP server runs inside the service -- simplest, but couples their lifecycles
2. **Sidecar:** a separate container beside the service, sharing its pod but deployed independently
3. **Centralized gateway:** one MCP server fronts many services, giving a single place for auth and audit
4. **Federated servers:** each domain runs its own MCP server, discovered through a shared registry
5. **Hybrid deployment:** a gateway for external callers with embedded or sidecar servers internally



KEY CONSIDERATIONS

- **Networking:** service discovery and secure service-to-service communication
- **Security:** run as non-root, minimal images, secure secrets management
- **State management:** keep containers stateless, externalize state to backing services

KEY TAKEAWAY Start simple with an embedded or sidecar server, then evolve toward a gateway or federated model as scale and governance needs grow. • Minimal images, non-root execution, externalized state, and health probes apply to an MCP server container exactly as they do to any Spring Boot service.

DEPLOYING MCP-ENABLED SERVICES TO OPENSIFT · MCP GATEWAY AND AGGREGATION PATTERNS



DEPLOYMENT STEPS

- 1. Build & push container image:** package the MCP-enabled service and publish it with an immutable tag
- 2. Create OpenShift resources:** declare the Deployment, Service, and Route that run and expose it
- 3.** Configure & secure (env, secrets, service accounts, network policies)
- 4. Deploy:** roll out to the cluster and let the probes confirm the pods are genuinely ready
- 5. Verify & test:** call each exposed tool and check the audit records land as expected
- 6. Monitor & scale:** watch latency, error rate, and tool usage, then size replicas to real demand



GATEWAY RESPONSIBILITIES

- Tool discovery and aggregation across multiple MCP servers
- AuthN/AuthZ, rate limiting, policy and guardrails, caching, observability



DESIGN TIP

- Start with a centralized gateway for simplicity; evolve to domain-based or federated as you scale

KEY TAKEAWAY Routes, ConfigMaps, Secrets, Service Accounts, and readiness probes give an MCP server the same enterprise-grade reliability as its backend services. • Centralized, domain-based, hierarchical, and federated gateway patterns each balance simplicity against isolation and team autonomy differently.

MCP VS. TRADITIONAL API INTEGRATION · MCP USE CASES FOR ENTERPRISE MICROSERVICES · MCP DESIGN ANTI-PATTERNS

📌 KEY DIFFERENCES

DIMENSION	TRADITIONAL API	MCP INTEGRATION
Discovery	Manual; client must know each endpoint	Built-in tool/resource discovery
Integration effort	High; custom code/SDK per service	Low; standard client works with any server
Change management	Client updates required for any API change	Tool updates/versioning handled by the server

📌 TOP MCP USE CASES (SELECTED)

- **AI-powered service operations:** restart a failed service, fetch logs, scale deployments
- **Customer support automation:** check order status, issue refund, update shipping address
- **Compliance and audit automation:** collect access logs, validate controls, generate reports

❌ TOP ANTI-PATTERNS

- Overly broad tools (doEverything()), leaky abstractions exposing internal IDs/SQL
- Chatty MCP servers, poor or inconsistent schemas, no authentication or authorization
- Stateful MCP servers, no rate limiting, ignoring observability, unbounded/unsafe actions

KEY TAKEAWAY Discovery, integration effort, schema consistency, and change management all favor MCP once an AI agent needs many tools across many teams. • Ten concrete use case patterns, each with standardization, security, discoverability, scalability, and reusability as shared benefits. • Each anti-pattern has a simple fix: narrow the tool, hide the internals, enforce auth, add limits, or keep the server stateless.

MCP PRODUCTION-READINESS CHECKLIST · DESIGNING MICROSERVICES FOR AI CONSUMPTION



PRODUCTION-READINESS AREAS

- Tool design, security and authorization, data minimization
- Schema management, observability, safe error handling, testing, governance

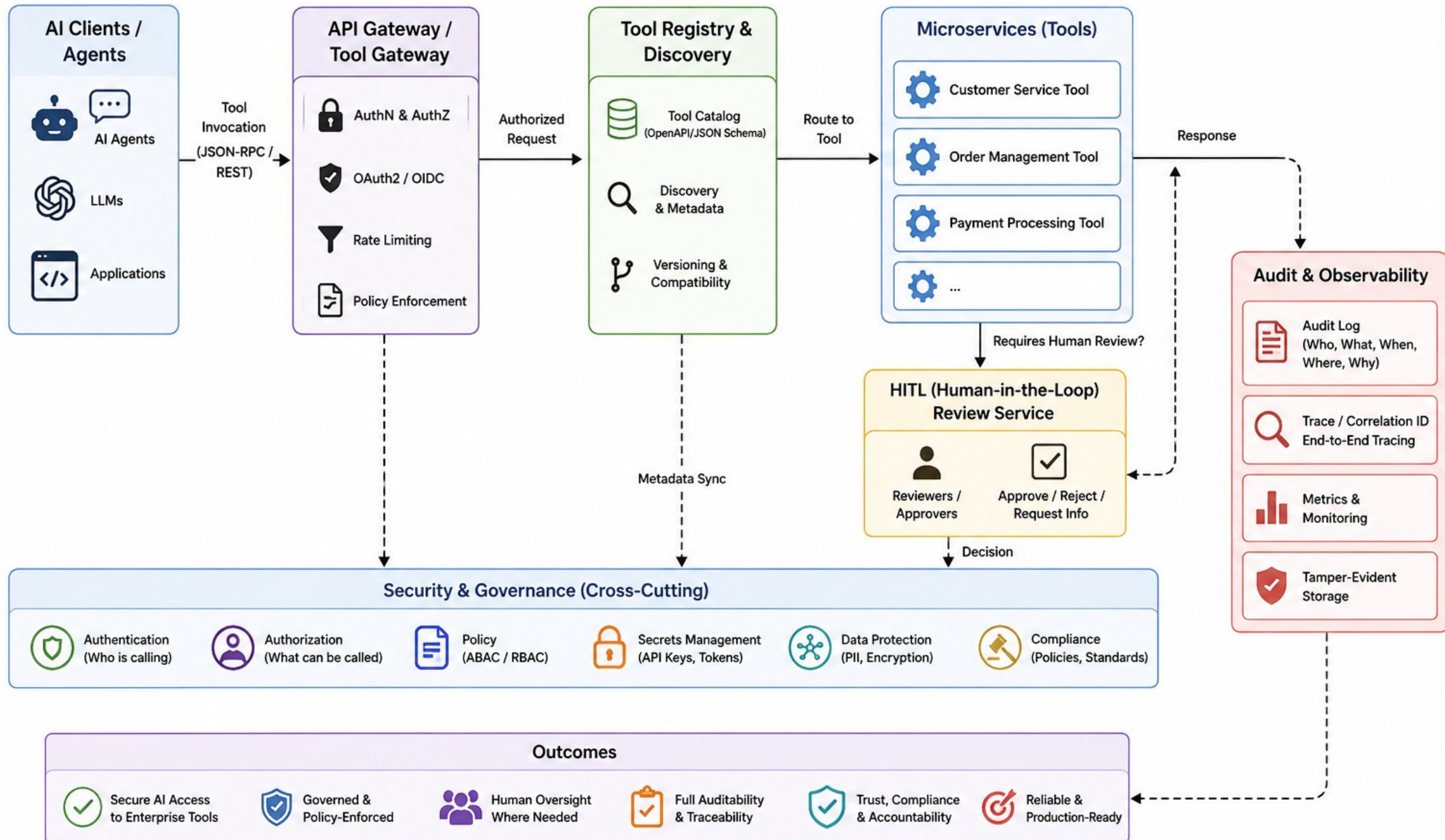


WHAT CHANGES FOR AI CONSUMPTION

- Descriptive, self-explanatory names and schemas replace implicit developer knowledge
- Bounded, predictable, idempotent operations replace loosely-specified endpoints

KEY TAKEAWAY MCP does not replace security, governance, or service ownership -- it adds a structured integration layer on top of all three. • An AI agent can't infer intent the way a human developer reading documentation can -- the API itself has to carry that clarity.

Exposing Microservices as AI-Callable Tools - Auth, HITL, Audit



Human API Consumers vs. AI Consumers

Different consumers, different expectations. Design APIs that work well for **both**.

 HUMAN API CONSUMERS	KEY DIFFERENCES	 AI API CONSUMERS
 People interact with clarity, context, and a purpose. They can adapt, ask for help, and handle variability.	Discovery  Browses documentation vs. Discovers via schemas/metadata	 AI agents need predictable, well-structured, and machine-readable interfaces to act safely and effectively.
 Explore and Understand Read docs, explore endpoints, understand options and tradeoffs.	Goal  Broad goals vs. Specific, task-oriented actions	 Discover via Schemas and Metadata Rely on OpenAPI/JSON Schema, tool metadata, and examples.
 Flexible and Adaptive Can adjust requests, handle errors, and retry with different inputs.	Reasoning  Humans infer and decide vs. AI follows patterns and instructions	 Deterministic and Predictable Expect consistent behavior, stable contracts, and bounded outcomes.
 Context-Rich Communication Can read long messages, handle complex instructions, and provide feedback.	Error Handling  Asks for help, tries alternatives vs. Expects clear, machine-readable errors	 Structured Input and Output Require strict schemas, concise instructions, and structured responses.
 Forgiving of Minor Issues Humans tolerate small inconsistencies and can recover from unclear responses.	Consistency  Handles some inconsistency vs. Needs consistent behavior	 Machine-Readable Errors Expect clear error codes, messages, and actionable remediation.
 Security via Identity Use strong authentication, roles, and policies to access APIs.	Data Consumption  Reads for information vs. Parses for action and structured output	 Least-Privilege Access Use scoped tokens, tool-level permissions, and explicit authorization.
 Optimized for Experience Focus on usability, helpful errors, and meaningful responses.		 Optimized for Automation Focus on reliability, idempotency, and efficiency for autonomous tasks.



KEY TAKEAWAY

Design APIs that are clear for humans and unambiguous for AI.
Provide rich documentation for people and strict contracts for machines.



Great APIs delight **both** people and agents.

DESCRIPTIVE TOOL NAMES AND SCHEMAS · BOUNDED AND PREDICTABLE TOOL BEHAVIOR · IDEMPOTENT AI-CALLABLE OPERATIONS



WHY DESCRIPTIVENESS MATTERS

- The model chooses which tool to call based largely on its name and description
- Ambiguous or generic names increase the risk of incorrect tool selection



WHAT BOUNDED BEHAVIOR MEANS

- A fixed, well-defined scope of effect for every tool invocation
- No surprising side effects beyond what the tool's name and schema describe



WHY IDEMPOTENCY MATTERS FOR AI TOOLS

- Agents may retry a call after an ambiguous or timed-out response
- An idempotent tool produces the same result whether called once or several times

KEY TAKEAWAY A clear, specific tool name and description reduces the chance an agent picks the wrong tool or misuses the right one. • Predictability lets an agent (and a human reviewer) reason confidently about what a tool call will actually do before it happens. • Idempotency keys and upsert-style operations protect against the retries and duplicate calls that are common in agentic workflows.

PREVENTING OVERLY POWERFUL TOOLS · READ-ONLY VS. STATE-CHANGING TOOLS · CONFIRMATION BEFORE DESTRUCTIVE ACTIONS



THE RISK OF OVERLY POWERFUL TOOLS

- A single tool with too many parameters or too broad a scope increases misuse risk
- Splitting into small, single-purpose, well-scoped tools reduces the blast radius of any mistake



TWO TOOL CATEGORIES

- **Read-only tools:** retrieve information, no side effects, lower risk
- **State-changing tools:** create, update, or delete data, requiring stronger controls



WHY CONFIRMATION MATTERS

- Irreversible operations (delete, cancel, reverse) carry outsized risk if triggered incorrectly
- A confirmation step, tied to human approval, prevents accidental or malicious execution

KEY TAKEAWAY Narrow, single-purpose tools limit the damage any one incorrect or malicious tool call can do. • Clearly labeling which tools only read and which tools change state is the first step toward applying the right level of control to each. • Adding a confirmation prompt for irreversible actions gives a human one last chance to catch an agent's mistake before it's permanent.

AI AGENT AUTHENTICATION AND AUTHORIZATION · POLICY ENFORCEMENT AT THE TOOL BOUNDARY · RATE LIMITS AND USAGE QUOTAS



AUTHENTICATING AN AI AGENT

- OAuth2 client credentials for the agent's own service identity
- Delegated user tokens when the agent acts on behalf of a specific human user



WHY THE BOUNDARY IS THE RIGHT PLACE

- Every tool call passes through the same enforcement point before execution
- Centralized policy is easier to audit, update, and reason about than logic scattered across tools



WHY RATE LIMITS MATTER FOR AI TOOLS

- Agent loops can call a tool far more frequently than typical human usage patterns
- Per-agent, per-tool, and per-tenant quotas contain the impact of a misbehaving agent

KEY TAKEAWAY Whether via OAuth2 client credentials, delegated user tokens, or service identity, the agent's calls must be authenticated exactly like any other client.

- Centralizing policy at the boundary means individual tool implementations don't each need to reimplement the same authorization logic.
- Quotas scoped per agent, per tool, or per tenant prevent one runaway agent loop from overwhelming a shared backend service.

AUDIT RECORDS FOR AGENT ACTIONS · CORRELATION ACROSS AGENT AND SERVICE CALLS · HANDLING AMBIGUOUS AGENT REQUESTS



WHAT AN AUDIT RECORD SHOULD CAPTURE

- Which agent, acting for which user, called which tool, with what arguments
- The outcome, timestamp, and any approval decision tied to the action



WHY CORRELATION MATTERS

- Ties together the agent's reasoning, tool calls, and downstream service calls as one story
- Essential for debugging, auditing, and understanding a multi-step agent workflow after the fact



HANDLING AMBIGUITY SAFELY

- Return a clear error or clarification request rather than guessing at intent
- Require explicit, unambiguous parameters for any state-changing tool

KEY TAKEAWAY Audit records for agent actions are the evidence trail that lets compliance, security, and support teams reconstruct exactly what happened and why. • Without shared correlation IDs, a single agent workflow's activity is scattered across dozens of unconnected log lines. • An agent that guesses at an ambiguous parameter and proceeds can take the wrong action confidently -- worse than simply asking for clarification.

DESIGNING EXPLAINABLE TOOL RESPONSES · COMPENSATING FOR INCORRECT AGENT ACTIONS



WHAT MAKES A RESPONSE EXPLAINABLE

- **Include supporting evidence:** source IDs, timestamps, and the specific data used
- State confidence or uncertainty explicitly rather than presenting every answer as certain



DESIGNING FOR COMPENSATION

- Define a compensating action for every state-changing tool before it goes live
- Compensation is a business action that restores an acceptable state, not always a perfect undo

KEY TAKEAWAY Including the reasoning, source data, or evidence behind a tool's answer is what lets a downstream summary be genuinely grounded, not just plausible. • Reversing a transaction, closing an incorrectly opened case, or issuing a correction notice are compensations designed in advance, not improvised after the fact.

TESTING AI-TO-MICROSERVICE INTEGRATION · EVALUATING TOOL SELECTION ACCURACY · MONITORING AI-DRIVEN SERVICE TRAFFIC



WHAT TO TEST END TO END

- Correct tool selection for a given natural-language request
- Correct parameter extraction, successful backend call, and correctly formatted final response



MEASURING TOOL SELECTION QUALITY

- Build a test set of realistic requests with known-correct tool selections
- Track selection accuracy over time as tools and descriptions evolve



WHAT TO MONITOR SEPARATELY

- Request volume and pattern specifically from AI agents versus human traffic
- Tool call error rates, retry patterns, and any anomalous usage spikes

KEY TAKEAWAY An agent, a tool, and a microservice can each pass their own unit tests while the integrated path between them still silently breaks. • Measuring how often an agent selects the correct tool for a given request is a direct, testable signal of tool design quality. • Tagging requests as agent-originated makes it possible to distinguish AI-driven load patterns from ordinary human traffic in every dashboard.

ENTERPRISE CAUTION AND SCOPE BOUNDARIES

Every capability in this module compounds risk when combined -- enterprise caution means growing agent scope deliberately, not all at once.

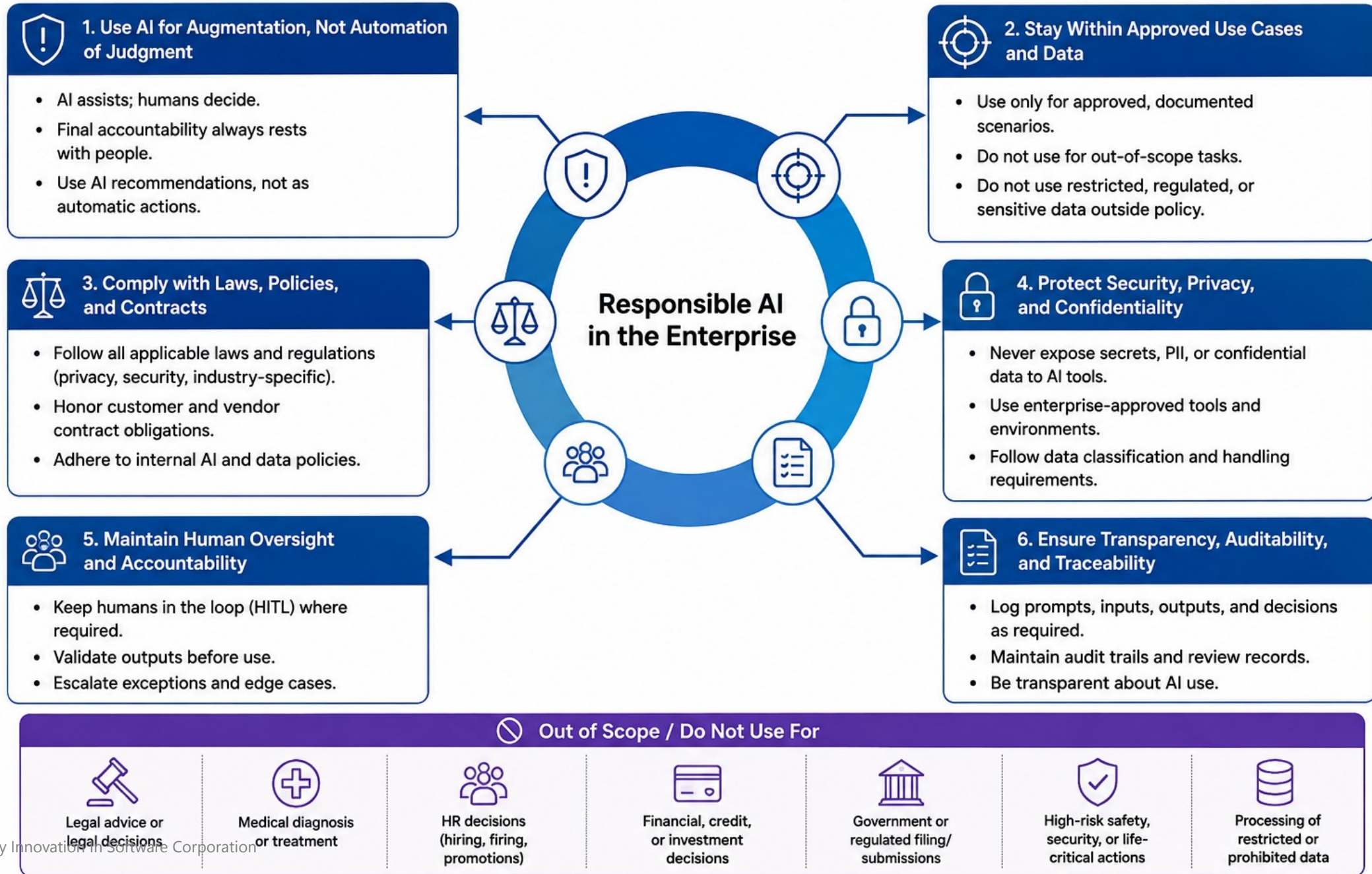


STAYING WITHIN SAFE SCOPE BOUNDARIES

- Start with read-only, low-risk tools and prove reliability before adding action tools
- Expand agent scope only with evidence of accuracy, safety, and business value

KEY TAKEAWAY Every capability in this module compounds risk when combined -- enterprise caution means growing agent scope deliberately, not all at once.

Enterprise Caution and Scope Boundaries



EXERCISE 5.3 — IDENTIFY MCP SECURITY CONTROLS FOR BANKING APIS

Module 12 checkpoint C — Auth, HITL, audit for tool exposure.



OBJECTIVE

- Auth, HITL, audit for tool exposure
- 15 min · worksheet checkpoint
- **Worksheet:**
labs/day-05/lab5/starter/mcp-controls.md



DO THIS

1. If get_assessment / submit_review were MCP tools, who may call them (authn/authz)?
2. Which tool requires human-in-the-loop before money-movement side effects?
3. What audit fields would you persist (actor, tool, correlation id, decision)?
4. Do not start an MCP process, Keycloak, or a Microsoft AI agent runtime



EXPECTED RESULT

- A completed MCP controls worksheet. Awareness only — no MCP runtime in the lab.
- Fill mcp-controls.md. You do not run an MCP server. Compare with solution/mcp-controls.md when asked.

KEY TAKEAWAY A completed MCP controls worksheet. Awareness only — no MCP runtime in the lab.